



Fakultät Informatik

**Entwicklung eines VHDL-basierten Applikationsbeispiels
zur Ablösung des FM350-2 Moduls durch das TM FAST
Modul**

Bachelorarbeit im Studiengang Informatik

vorgelegt von

Jan-Eric Gedicke

Matrikelnummer 3643446

Erstprüfer:

Prof. Dr. Meitner

Zweitprüfer:

Prof. Dr. Albrecht

Betreuer im Unternehmen:

Conrad, Maximilian

Unternehmen:

Siemens AG

© 2025

Inhaltsverzeichnis

1	Einleitung	1
1.1	Herausforderung	1
1.2	Zielsetzung der Arbeit	2
2	Grundlagen	3
2.1	Überblick über SIMATIC S7	3
2.2	Technologiemodule	6
2.3	Software	7
2.4	Programmierung einer PLC mit TIA-Portal	10
2.5	Grundlagen zu Inkrementalgebern	11
3	Analyse der Fähigkeiten des FM350-2 und dem TM FAST Modul	12
3.1	Funktionsweise des FM350-2	12
3.2	Eigenschaften und Aufbau des TM FAST	21
3.3	Direkter Vergleich der TM FAST und FM 350-2	25
4	Entwicklung des VHDL-basierten Applikationsbeispiels	27
4.1	Anforderungen an die Neuentwicklung	27
4.2	Umsetzung der funktionalen Anforderungen	29
4.3	Umsetzung der nicht-funktionalen Anforderungen	36
4.4	Validierung und Tests	43
5	Fazit und Ausblick	46
5.1	Kritische Reflexion und Herausforderungen	47
5.2	Potenzielle Weiterentwicklungen und zukünftige Anwendungen	47

1 Einleitung

Die Produktpalette der Firma Siemens im Bereich der Industrieautomatisierung umfasst verschiedene Steuerungs- und Erweiterungsmodule, von denen die Produktserie SIMATIC S7-300 schrittweise abgekündigt wurde [25]. Insbesondere das FM350-2 Modul, das für Dosier- und Zähl Anwendungen entwickelt wurde, ist nicht mehr in der aktiven Vermarktung und lediglich als Ersatzteil verfügbar. Das FM350-2 Modul bietet 8 Kanäle für hochkanalige Dosieranwendungen. Unter hochkanaligen Dosieranwendungen versteht man Systeme, bei denen zahlreiche unterschiedliche Materialströme (z. B. Flüssigkeiten, Pulver oder Granulate) gleichzeitig präzise gesteuert, verwogen und dosiert werden müssen. Solche Systeme finden vor allem in der Chemie-, Pharma- oder Lebensmittelindustrie Anwendung, da dort eine große Anzahl paralleler Dosierstellen eine hohe Effizienz und Flexibilität erfordert [26].

Ein alternatives Lösungskonzept ist der Einsatz des TM FAST Moduls, eines frei-programmierbaren Hochgeschwindigkeits-Moduls, das eine erweiterte Kanalanzahl ermöglichen kann und für die neuere Produktserie SIMATIC S7-1500 ausgelegt ist [3]. Ziel dieser Bachelorarbeit ist die Entwicklung eines VHDL-basierten Applikationsbeispiels zur Demonstration der Einsatzmöglichkeiten des TM FAST Moduls in hochkanaligen Dosieranwendungen. Gleichzeitig soll die Integration in das TIA Portal erfolgen, um eine reibungslose Nutzung in industriellen Automatisierungsumgebungen zu gewährleisten.

1.1 Herausforderung

In der Automatisierungstechnik ist es entscheidend, dass hochkanalige Dosierprozesse zuverlässig, präzise und mit minimaler Verzögerung gesteuert werden können. Für das hier betrachtete System, bestehend aus dem TM FAST Modul in Verbindung mit einer SIMATIC S7-1500 PLC, ist jedoch unklar, ob die bestehenden Anforderungen des FM350-2 Moduls hinsichtlich Genauigkeit, Zykluszeit und

Synchronisation vollständig erfüllt werden können. Dies betrifft insbesondere die Fähigkeit, mehrere Dosierkanäle gleichzeitig mit hoher Geschwindigkeit und minimalen Verzögerungen zu verarbeiten, sowie die effiziente Nutzung der verfügbaren FPGA-Ressourcen bei gleichzeitig erweiterter Kanalanzahl.

1.2 Zielsetzung der Arbeit

Ziel dieser Bachelorarbeit ist die Entwicklung eines VHDL-basierten Applikationsbeispiels, das das FM350-2 Modul der SIMATIC S7-300 durch das TM FAST Modul der S7-1500 ersetzt.

Die Arbeit verfolgt insbesondere folgende Ziele:

- Umsetzung der bekannten FM350-2-Funktionalitäten auf dem TM FAST Modul.
- Erhöhung der Kanalanzahl von 8 auf die höchstmögliche Anzahl für flexible Dosieranwendungen.
- Optimierung der Implementierung hinsichtlich **Performance** und **Ressourcenschonung**, z. B. Maximalauslastung der Logikelemente von 80 % und Senkung der Zykluszeit von 50 μ s auf 20 ns.
- Integration des Applikationsbeispiels in das TIA Portal zur nahtlosen Nutzung in industriellen Automatisierungslösungen.

Die erfolgreiche Umsetzung dieser Ziele soll die Grundlage für den Einsatz des TM FAST Moduls in hochkanaligen Dosieranwendungen schaffen und die Vorteile moderner FPGA-Technologie in der industriellen Automatisierung demonstrieren.

2 Grundlagen

Um die technischen Aspekte und die Funktionsweise der in dieser Arbeit behandelten Systeme zu verstehen, werden im Folgenden die zugrundeliegenden Konzepte und Technologien erläutert. Relevant sind hierbei die SIMATIC S7, Speicherprogrammierbare Steuerungen, Quartus und Questa sowie dem TIA Portal.

2.1 Überblick über SIMATIC S7

Speicherprogrammierbare Steuerungen (SPS), im Englischen *Programmable Logic Controllers* (PLC), wie die SIMATIC S7-Serie von Siemens, übernehmen Steuerungs- und Regelungsaufgaben in Maschinen und Anlagen. Sie können unter anderem Antriebe, Ventile, Pumpen und Sensoren ansteuern und zeichnen sich durch ihre freie Programmierbarkeit sowie ihre universelle Einsetzbarkeit aus [6]. Im Folgenden werden Aufbau, Funktionsweise und zentrale Komponenten eines SIMATIC S7-Systems erläutert.

2.1.1 Aufbau eines SIMATIC S7-Systems

Ein SIMATIC S7-System besitzt einen modularen Aufbau und besteht aus mehreren Baugruppen mit spezifischen Funktionen. In der Abbildung 2.1 sind beispielhaft folgende Baugruppen zu erkennen: das PM/PS-Modul (Stromversorgung), die CPU (Zentraleinheit) sowie mehrere Ein- und Ausgangsmodule (I/O-Module) zur Verbindung mit Sensoren und Aktoren.

Controller (CPU)

Die CPU bildet die zentrale Verarbeitungseinheit der Steuerung. Ihre Hauptaufgabe besteht in der Ausführung des Anwenderprogramms, das zur Umsetzung spezifischer Steuerungs- und Regelungsaufgaben dient [6]. Sie verarbeitet die Signale

der Ein- und Ausgabebaugruppen, führt Berechnungen durch und gibt Steuerbefehle an die angeschlossenen Sensoren und Aktoren weiter. Darüber hinaus verfügen SIMATIC S7-Controller über integrierte Kommunikationsschnittstellen, wie Ethernet/PROFINET oder PROFIBUS, die einen Datenaustausch mit weiteren Steuerungen, Bediengeräten oder übergeordneten Systemen (z. B. SCADA oder MES) ermöglichen [15].

Signal- und Technologiebaugruppen

Signalbaugruppen (Eingangs- und Ausgangsbaugruppen) stellen die Verbindung zur Maschine oder Anlage her. Sie erfassen digitale oder analoge Prozesssignale (z. B. Spannungen, Ströme) und geben diese an die CPU weiter bzw. leiten Signale an das Feld zurück.

Technologiebaugruppen (*Technologiemodule*) erweitern die Steuerung um hardwaregestützte Funktionen. Sie verfügen über eigene Prozessoren oder FPGAs und können dadurch zeitkritische Aufgaben (z. B. Hochgeschwindigkeitszählen, präzise Zeitstempel, Frequenz- und Pulsmessungen, Positionserfassung oder schnelle Reaktionsausgänge) unabhängig und deterministisch ausführen. Auf diese Weise werden die durch die CPU-Zykluszeit bedingten Latenzen umgangen, die CPU entlastet und die Systemperformance insgesamt gesteigert [5].

Kommunikationsbaugruppen

Kommunikationsmodule erweitern die Schnittstellen- und Protokollfähigkeit einer Steuerung. Sie ermöglichen beispielsweise zusätzliche Feldbus- oder Industrial-Ethernet-Anbindungen und kapseln die entsprechenden Kommunikationsstacks, wodurch die CPU weiter entlastet wird [5].

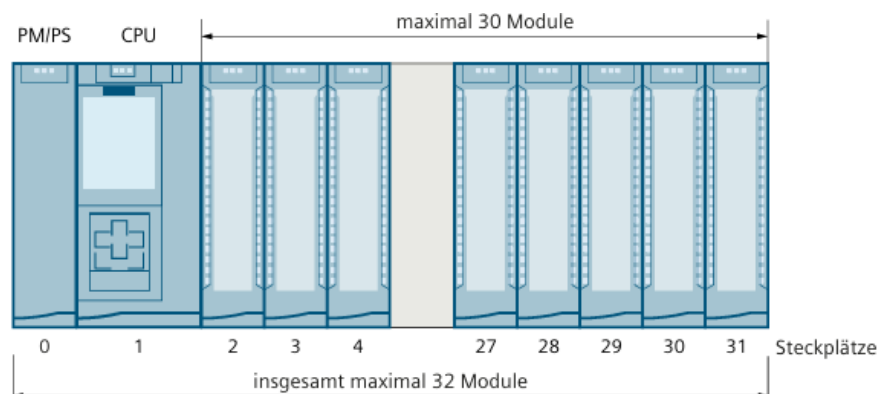


Abbildung 2.1: Aufbau eines SIMATIC S7-Systems [9]

2.1.2 Funktionsweise und Aufbau einer SPS

Eine Speicherprogrammierbare Steuerung (SPS) arbeitet nach dem EVA-Prinzip (Eingabe–Verarbeitung–Ausgabe). Über Eingangsbaugruppen werden die Zustände von Sensoren (z. B. Endschalter, Temperaturfühler, Drehzahlsensoren) erfasst und in einem *Prozessabbild der Eingänge* zwischengespeichert. Dieses Abbild dient als Grundlage für die Bearbeitung durch das Anwenderprogramm. Nach der Verarbeitung werden die Ergebnisse in das *Prozessabbild der Ausgänge* geschrieben und an die Ausgangsbaugruppen übertragen, wodurch die Aktoren (z. B. Motoren, Ventile) angesteuert werden. Dieser Zyklus wiederholt sich kontinuierlich im Millisekundenbereich [16, 6].

Das Anwenderprogramm ist modular aufgebaut und besteht aus *Organisationsbausteinen* (z. B. OB1 als Hauptzyklus), *Funktionsbausteinen* (FB), *Funktionen* (FC) und *Datenbausteinen* (DB). OB1 kann durch höher priorisierte Bausteine wie Interrupt- oder Alarm-OBs unterbrochen werden, wodurch sich die Zykluszeit verlängern kann. Überschreitet die Zykluszeit eine definierte maximale Zykluszeit (Watchdog), geht die SPS in einen Fehlerzustand und beendet die Abarbeitung des Programms (Stopp-Zustand), und alle Ausgänge werden auf 0 gesetzt [17].

Eine modulare SPS besteht mindestens aus einem Baugruppenträger, einer Stromversorgung, einer CPU mit Speicher sowie Ein-/Ausgabebaugruppen. Kompakt-SPS vereinen diese Komponenten in einem Gehäuse. Optional können Kommunikations-, Zähler-, Positionier- oder Regelungsmodule ergänzt werden. Mehrere Baugruppenträger werden über Interface-Module verbunden [18].

Durch diese Struktur ist die SPS in der Lage, deterministisch und zuverlässig auf geänderte Eingangssignale zu reagieren, Prozesse zu steuern und eine hohe Betriebssicherheit zu gewährleisten [18]. Die Bearbeitung erfolgt dabei im Wesentlichen zyklisch, indem das Anwenderprogramm in festen Zyklen durchlaufen wird. Ergänzend können azyklische Dienste, beispielsweise ereignis- oder zeitgesteuerte Organisationsbausteine (OBs), spezifische Aufgaben außerhalb des regulären Zyklus übernehmen [5, 1].

2.2 Technologiemodule

Die SIMATIC S7-1500 Steuerung ist ein modular aufgebautes System, dessen Funktionalität flexibel skalierbar ist. Zentraler Bestandteil ist die S7-1500 CPU, in der das Anwenderprogramm ausgeführt wird und die Schnittstellen zu weiteren Automatisierungskomponenten bereitstellt. Neben der CPU lässt sich das System durch verschiedene Module erweitern, darunter Interface-, Signal-, Kommunikations- und Technologiebaugruppen. Das Peripheriesystem ET 200MP ermöglicht den Einsatz der S7-1500 IO-Module entweder direkt an der Steuerung oder über einen kompatiblen PROFINET-/PROFIBUS-Teilnehmer wie das Interfacemodul IM 155 PN, das den Datenaustausch zwischen Steuerung und Peripheriemodulen übernimmt. Abbildung 2.2 zeigt links den zentralen Aufbau mit CPU und vier Signalbaugruppen und rechts einen vergleichbaren dezentralen Aufbau mit Interfacemodul. Die Kommunikation erfolgt hierbei über PROFINET. Die Signalbaugruppen umfassen sowohl digitale als auch analoge Ein- und Ausgänge, die der Steuerung zusätzliche Funktionalität bereitstellen [9].



Abbildung 2.2: Zentraler und dezentraler Aufbau der SIMATIC S7-1500 Steuerung mit ET 200MP Peripheriesystem [9]

Technologiebaugruppen bieten eine hardwarenahe Signalvorverarbeitung für schnelle Zähl- und Messaufgaben sowie präzise Positionserfassung mittels Inkremental- oder SSI-Absolutwertgebern. Bisherige Module wurden über Technologieobjekte oder direkt im Anwenderprogramm im TIA Portal parametrierbar und programmiert. Das aktuelle Portfolio beinhaltet unter anderem das TM NPU, TM Count 2x24 V, TM Posinput 2, TM Timer DIDQ 16x24 V und das TM PTO 4. Das TM

NPU arbeitet mit einem trainierten neuronalen System und erlaubt den Anschluss verschiedener Sensoren über Gigabit-Ethernet oder USB 3.1. Die Zählerbaugruppe TM Count 2x24 V ist vielseitig einsetzbar für Zähl- und Messaufgaben sowie zur Positionserfassung über Inkrementalgeber. Das TM Posinput 2 Modul stellt zwei Zählkanäle für Differenzeingangssignale nach RS422 mit bis zu 1 MHz bereit und ermöglicht die Positionserfassung mit SSI-Absolutwertgebern. Der TM Timer DIDQ 16x24 V dient zeitkritischen Aufgaben, wie z. B. der präzisen Einhaltung von Reaktionszeiten. Das PTO 4 Modul unterstützt die Ansteuerung von Schrittmotoren und Servoantrieben mit PTI-Schnittstelle [3].

Das TM FAST Modul ist eine FPGA und dadurch freiprogrammierbar. Es ist bereits länger Bestandteil des Portfolios und wird kontinuierlich um neue Funktionen erweitert, die aus konkreten Kundenanwendungsfällen abgeleitet werden, wie zusätzliche Protokoll-, Zähl-, Trigger- und Preprocessing-Bausteine. Diese Funktionen werden als wiederverwendbare Funktionsblöcke bereitgestellt, wodurch bestehende Anforderungen schneller umgesetzt und zukünftige Anwendungsfälle flexibel ergänzt werden können [11].

2.3 Software

Für die Erstellung einer Anwendung auf dem TM FAST werden mehrere Softwareprogramme benötigt. Das TIA Portal übernimmt die Hardwarekonfiguration sowie die Programmierung des Anwenderprogramms, das die CPU steuert und den Funktionsumfang des TM FAST aktiviert. Über das Engineering-Framework kann zudem die Firmware auf das Modul geladen werden. Für die Entwicklung auf dem integrierten Intel® FPGA Cyclone 10 LP wird die Designumgebung Intel® Quartus® Prime Lite eingesetzt. Zur Simulation und Verifikation des Hardwareentwurfs kommt die Questa-Intel® FPGA Starter Edition zum Einsatz.

2.3.1 Totally Integrated Automation Portal

Das *Totally Integrated Automation Portal* (TIA Portal) ist die zentrale Engineering-Plattform für SIMATIC S7-Steuerungen. Es ist das zentrale Tool für die Projektierung, Programmierung, Inbetriebnahme und Diagnose von Automatisierungsprojekten und deckt damit sowohl die Hardwarekonfiguration als auch die Softwa-

reentwicklung ab [1]. Die Umgebung umfasst u. a. SIMATIC STEP 7 für die SPS-Programmierung, SIMATIC WinCC für HMI-Anwendungen, SINAMICS Start-drive zur Antriebskonfiguration sowie die SIMATIC Energy Suite für Energiemonitoring [7, 8].

In dieser Arbeit wird TIA Portal V20 verwendet. Für die Kommunikation zwischen CPU und TM FAST ist eine entsprechende Hardwarekonfiguration erforderlich. Hierbei werden die Komponenten anhand ihrer Artikelnummer ausgewählt und im Projekt verknüpft. Abb. 2.3 zeigt eine Beispielkonfiguration bestehend aus Powermodul (PM), CPU 1516-3 PN/DP und TM FAST. Die Baugruppen sind über den Rückwandbus verbunden, wodurch eine zusätzliche PROFINET- oder PROFIBUS-Verbindung ist nicht notwendig.



Abbildung 2.3: Hardwarekonfiguration im TIA Portal mit TM FAST

Nach der Konfiguration erhält das TM FAST im Projekt eine eigene Ansicht und wird als Teil des gesamten Aufbaus dargestellt. Unter *Online & Diagnostics* können Modulstatus, Diagnoseinformationen und Firmwareupdates verwaltet werden. Über *Commissioning* lassen sich die aktuelle User-Logik und weitere Kommandos wie das Laden der Anwendung aus dem Flash-Speicher oder die Aktivierung der Debug-Schnittstelle aufrufen. Für die azyklische Kommunikation stehen die TM FAST User Data Records zur Verfügung, die manuell oder über Funktionsbausteine beschrieben und gelesen werden können. Die dafür bereitgestellte TIA-Bibliothek LTMFAST enthält unter anderem die Anweisungen LTMFAST ControlREC (Ist für die Steuerung des TM FAST zuständig), UserReadREC (Liest Daten aus dem TM FAST), UserWriteREC (Schreibt Daten in den TM FAST) und AppDownload (Lädt die Anwendung auf den TM FAST) [7].

2.3.2 Intel® Quartus® Prime

Intel® Quartus® Prime ist eine Entwicklungsumgebung für FPGAs. Sie deckt den gesamten Prozess von der Designerfassung über die Synthese und Optimierung bis hin zur Simulation und Programmerstellung ab. Es existieren drei Editionen – Lite, Standard und Pro – die sich im Funktionsumfang und in der Geräteunterstützung unterscheiden. Für das Cyclone 10 LP im TM FAST wird in dieser Arbeit die Version 25.1.1 Lite Edition eingesetzt [21].

Ein Quartus-Projekt kann in mehrere Revisionen aufgeteilt werden, die jeweils eine eigene VHDL-Architektur und Konfigurationsdateien beinhalten. Diese Struktur erlaubt es, verschiedene Anwendungen unabhängig voneinander zu entwickeln. Die Kompilation umfasst die Schritte Analyse/Synthese, Fitter und Assembler:

- **Analyse und Synthese:** Überprüfung der logischen Konsistenz, Erzeugung von Zustandsautomaten, Flipflops und Logikzellen aus dem VHDL-Code, sowie Optimierung zur Ressourcenschonung.
- **Fitter:** Zuweisung der Logikfunktionen zu den FPGA-Ressourcen unter Berücksichtigung von Timing- und Routing-Anforderungen.
- **Assembler:** Erzeugung der Ausgabedateien wie `.sof`, `.pof` oder `.rbf`, die zur Programmierung des FPGAs genutzt werden [20].

2.3.3 Questa-Intel® FPGA

Zur Simulation von Hardwareentwürfen stehen zwei Versionen zur Verfügung: die kostenpflichtige Questa-Intel® FPGA sowie die Starter Edition. Beide basieren auf der Siemens EDA Questa Core Software und bieten umfangreiche Funktionen zur Simulation, Verifikation und zum Debugging von FPGA-Designs [22].

Die Simulation erfolgt entweder direkt aus Quartus oder über Simulationsskripte in Questa. Hierzu wird eine Testbench erstellt, die Eingabesignale bereitstellt und die resultierenden Ausgangssignale überprüft. Typischerweise werden dabei Prozesse definiert, die z. B. ein Taktsignal generieren oder Signalwechsel nachbilden. Abb. 2.4 zeigt eine beispielhafte Simulation zur Validierung einer Signum-Funktion [22].

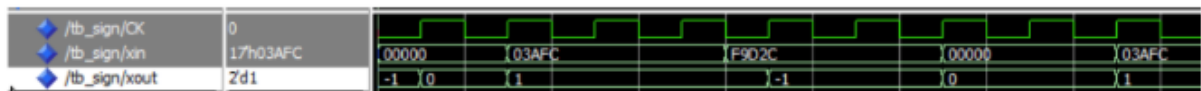


Abbildung 2.4: Zeitdiagramm zur Simulation einer Signum-Funktion in der Questa-Intel® FPGA Starter Edition

2.4 Programmierung einer PLC mit TIA-Portal

Die Grundlage für das *Totally Integrated Automation Portal* (TIA Portal) bildet die Norm IEC 61131-3, die einen einheitlichen Standard für die Softwareentwicklung in speicherprogrammierbaren Steuerungen (SPS) definiert. Diese Norm legt die Struktur und die fünf standardisierten Programmiersprachen fest, wodurch eine herstellerübergreifende Programmierbarkeit ermöglicht wird [6].

Die fünf in IEC 61131-3 beschriebenen Sprachen sind: *Kontaktplan* (KOP), eine grafische Darstellung, die sich an Schaltplänen orientiert; *Funktionsplan* (FUP), eine grafische Sprache zur logischen Verknüpfung von Bausteinen; *Anweisungsliste* (AWL), eine textbasierte Sprache ähnlich zu Assembler; *Strukturierter Text* (ST), eine hochsprachähnliche Programmiersprache vergleichbar mit Pascal; sowie die *Ablaufsprache* (AS) zur Modellierung von Ablaufsteuerungen [6].

Funktionsplan (FUP)

FUP ist eine grafische Programmiersprache, in der logische Verknüpfungen und Operationen durch Symbole dargestellt und miteinander verbunden werden. Typische Anwendungen sind logische Grundfunktionen wie UND-, ODER- oder NICHT-Verknüpfungen. Darüber hinaus können auch komplexere mathematische Operationen oder steuerungsspezifische Aufgaben umgesetzt werden. FUP eignet sich besonders, um Programme übersichtlich und intuitiv zu strukturieren [5].

Strukturierter Text (ST)

SCL ist eine textbasierte Sprache, die in der IEC 61131-3 als *Strukturierter Text* bezeichnet wird. Sie ähnelt der Hochsprache Pascal und ist besonders für komplexe Algorithmen, Berechnungen und strukturierte Programmabläufe geeignet. Mit Schleifen, Verzweigungen und Funktionsaufrufen ermöglicht SCL eine sehr flexible und modulare Programmierung [6].

2.5 Grundlagen zu Inkrementalgebern

Inkrementalgeber erzeugen aus einer mechanischen Weg- oder Drehbewegung zwei zueinander um 90° phasenverschobene, rechteckförmige Ausgangssignale (A/B; Quadratur). Bei konstanter Geschwindigkeit entstehen gleichförmige Pulszüge; die Pulsfrequenz ist proportional zur Geschwindigkeit und kann über einen geeigneten Skalenfaktor in Geschwindigkeitswerte umgerechnet werden [13].

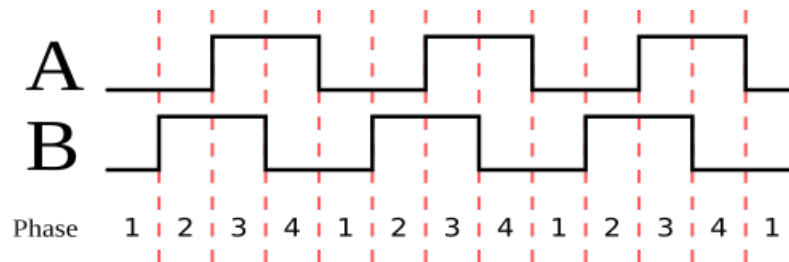


Abbildung 2.5: Zwei Rechteckwellen in Quadratur [12]

Rotative Geber realisieren dies typischerweise mit einer Lichtquelle, einer codierten Scheibe auf der Welle und Photodetektoren. Die Bewegungsrichtung wird aus der Phasenlage der Quadratur-Signale bestimmt: In einer Drehrichtung führt A vor B, in der anderen führt B vor A. Für eine höhere Auflösung kann nicht nur pro Puls, sondern auf jeder Flanke gezählt werden (1x-, 2x- oder 4x-Auswertung); dadurch lässt sich die Zählauflösung auf das bis zu Vierfache erhöhen [12, 13]. Aufgrund dieser Eigenschaften werden Inkrementalgeber häufig in Anwendungen eingesetzt, die eine präzise Messung und Regelung von Position und Geschwindigkeit erfordern.

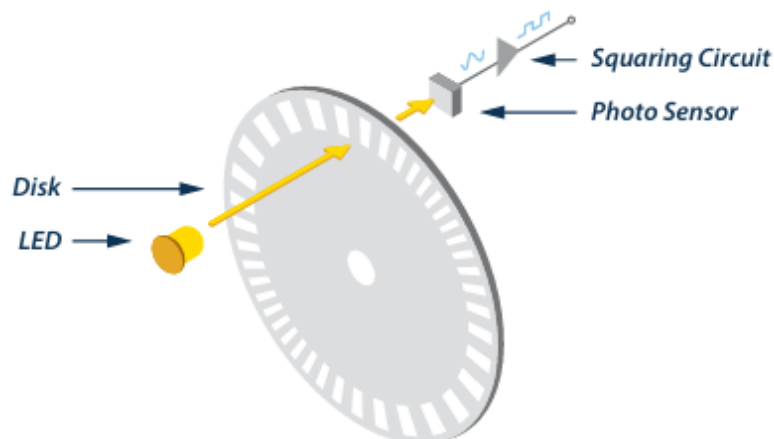


Abbildung 2.6: Funktionsweise eines Inkrementalgebers [12]

3 Analyse der Fähigkeiten des FM350-2 und dem TM FAST Modul

In diesem Abschnitt werden die technischen Grundlagen und Eigenschaften der beiden betrachteten Module, FM350-2 und TM FAST, vorgestellt. Es werden deren Aufbau, Funktionsweise, Schnittstellen und typische Einsatzgebiete erläutert. Ziel ist es, die relevanten Technologien und Unterschiede beider Systeme darzustellen, um die Basis für die spätere Analyse der Ablösung des FM350-2 durch das TM FAST Modul zu schaffen.

3.1 Funktionsweise des FM350-2

Die Funktionsbaugruppe FM 350-2 ist eine 8-kanalige Zählerbaugruppe mit integrierter Dosierfunktion für das Automatisierungssystem S7-300. Sie wird vor allem in Bereichen eingesetzt, in denen Signale gezählt und schnelle Reaktionen auf bestimmte Zählerstände notwendig sind. Besonders relevant ist die FM 350-2 für Anwendungen in der Fertigungs- und Automatisierungstechnik, wie beispielsweise in Verpackungs-, Sortier- und Dosieranlagen sowie bei der Drehzahlregelung und Überwachung von Gasturbinen.

Sie unterstützt einen maximalen Zählbereich von -2^{31} bis $+2^{31} - 1$ (entsprechend $-2\,147\,483\,648$ bis $+2\,147\,483\,647$) und erreicht je nach verwendeter Gebersignale eine maximale Eingangsfrequenz von bis zu 20 kHz pro Kanal [2].

Die Baugruppe ermöglicht verschiedene Betriebsarten: endloses, einmaliges sowie periodisches Zählen (jeweils vorwärts oder rückwärts) und Frequenzmessung. Der Zählvorgang kann wahlweise über das Anwenderprogramm (Software-Tor) oder über externe Signale (Hardware-Tor) gestartet und gestoppt werden. Dabei können Zähl-, Tor- und Richtungssignale direkt an die Baugruppe angeschlossen werden.

Für jeden Zählkanal ist es möglich, einen Vergleichswert zu definieren. Im Dosiermodus lassen sich bis zu vier Vergleichswerte je Kanal verwenden. Wird der definierte Vergleichswert erreicht, kann automatisch ein zugeordneter Ausgang gesetzt oder zurückgesetzt werden, um direkt steuerungsrelevante Aktionen auszulösen oder Prozessalarme zu generieren.

Innerhalb der Betriebsarten „Einmalig zählen“ und „Periodisch zählen“ lassen sich zusätzlich Zählgrenzen innerhalb des maximalen Zählbereichs festlegen. Im Vorwärtsbetrieb beginnt die Zählung bei 0, der Endwert ist dabei frei wählbar. Im Rückwärtsbetrieb wird der Startwert definiert, während der Endwert bei 0 liegt.

Die FM 350-2 unterstützt pro Zählkanal bis zu vier Prozessalarme. Zwei davon können durch Flankenwechsel am Hardware-Tor ausgelöst werden, zwei weitere sind abhängig von der jeweils eingestellten Betriebsart. Im Dosierbetrieb können bis zu fünf spezifische Alarmerzeuger erzeugt werden.

Für die Zählerfunktion können Signale unterschiedlicher Gebertypen verwendet werden. Zulässig sind ausschließlich prellfreie Geber, darunter 24-V-Inkrementalgeber (Gegentakt oder P-Schalter), 24-V-Impulsgeber mit Richtungspegel, 24-V-Initiatoren ohne Richtungspegel (z. B. Lichtschranken oder BERO Typ 2) sowie NAMUR-Geber nach DIN 19234. Die Signaltypen können an den Eingängen in Vierergruppen zwischen 24-V- und NAMUR-Signalen konfiguriert werden. Zu beachten ist, dass an für NAMUR konfigurierte Eingänge keine Signalspannungen über 8,2 V angelegt werden dürfen. An den Tor- (logische Freigabeeingänge, die den Zählpfad öffnen oder sperren) und Richtungseingängen sind ausschließlich 24-V-Signale erlaubt.

Zum Schutz vor Störungen sind alle Eingänge mit einem einheitlichen Eingangsfiler (RC-Glied) mit einer Filterzeit von 50 µs versehen. Schnelle Reaktionen auf Zählereignisse sind durch digitale Ausgänge realisierbar, wobei pro Kanal ein Ausgang und im Dosiermodus bis zu vier Ausgänge verfügbar sind. Diese können zählerstandsabhängig oder über Steuerbits angesteuert werden.

Das Verhalten der Baugruppe bei einem CPU-STOP ist parametrierbar. Es kann festgelegt werden, ob die Betriebsart fortgeführt oder abgebrochen wird und ob die digitalen Ausgänge ihren aktuellen Zustand beibehalten, auf Ersatzwerte gesetzt oder abgeschaltet werden. Dabei ist besondere Vorsicht geboten, da Ersatzwerte auch an nicht freigegebenen Ausgängen anliegen können, was unter Umständen zu gefährlichen Anlagenzuständen führt.

Auch bei Ausfall der Baugruppenversorgung zeigt die FM 350-2 ein differenziertes Verhalten. Wird sie über einen Standard-Rückwandbus betrieben, erkennt die CPU bei Spannungsverlust einen Peripherie-Zugriffsfehler, und die Baugruppe startet nach Spannungswiederkehr nicht automatisch neu. Beim Einsatz eines aktiven Rückwandbusses hingegen wird der CPU ein Ziehen-Alarm und bei Wiederkehr der Versorgungsspannung ein Stecken-Alarm gemeldet.

Zusätzlich verfügt die FM 350-2 über umfassende Diagnosefunktionen. Mögliche Fehlerzustände wie fehlerhafte Gebersversorgung (NAMUR), fehlende oder fehlerhafte Parametrierung, Zeitüberwachungsfehler (Watchdog), Drahtbruch oder Kurzschluss sowie der Verlust von Prozessalarmen können erkannt und gemeldet werden [2].

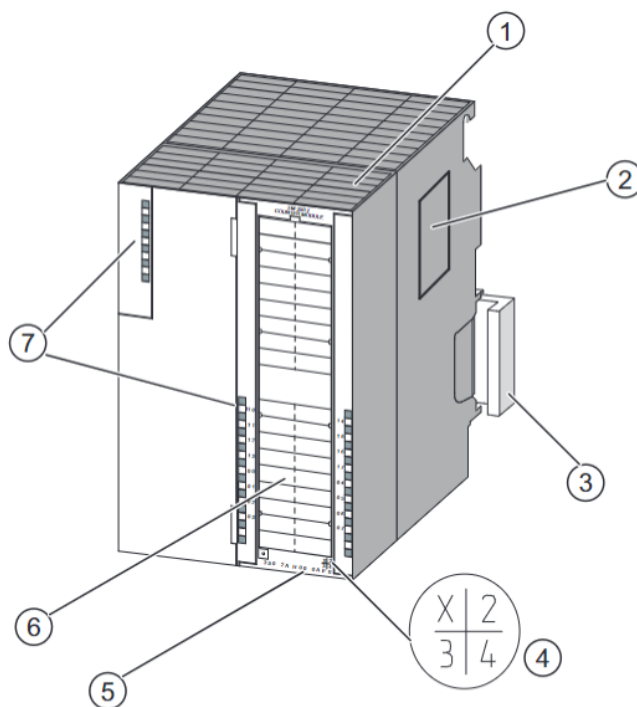


Abbildung 3.1: Hardware der FM-350-2 [2]

Nummer	Bezeichnung
1	Frontstecker
2	Typenschild
3	Busverbinder SIMATIC-Schnittstelle
4	Erzeugnisstand
5	Bestellnummer
6	Beschriftungsstreifen
7	Diagnose-LED
8	Status-LEDs

Tabelle 3.1: Kennzeichnungen und Komponenten der FM 350-2 [2]

3.1.1 Endloses Zählen bei der FM 350-2

Die Betriebsart „Endloses Zählen“ ermöglicht es der FM 350-2, Zählimpulse kontinuierlich zu erfassen, ohne dass der Zähler bei Erreichen einer Grenze stoppt. Wird beim Vorwärtszählen die obere Zählgrenze (+2 147 483 647) erreicht und ein weiterer Zählimpuls empfangen, springt der Zähler automatisch auf die untere Zählgrenze (−2 147 483 648) und zählt weiter. Analog dazu springt der Zähler beim Rückwärtszählen von der unteren Zählgrenze auf die obere Zählgrenze, sobald ein weiterer Zählimpuls erfolgt. Dadurch wird ein endloses Zählen innerhalb des 32-Bit-Zählbereichs realisiert.

Der Zählbereich ist fest auf 31 Bit vorzeichenbehaftet (−2 147 483 648 bis +2 147 483 647) und kann nicht verändert werden. Nach einem Neustart beginnt der Zähler bei 0.

Wurde ein Vergleichswert parametrieren, kann bei Erreichen des Vergleichswerts ein Prozessalarm ausgelöst und/oder ein Ausgang geschaltet werden.

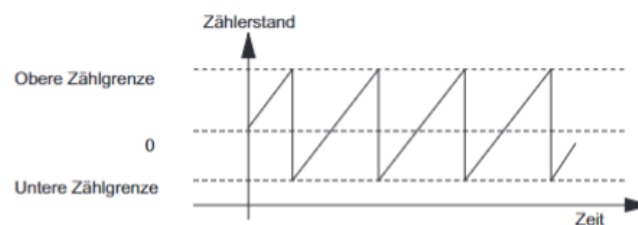


Abbildung 3.2: Endloses Zählen in Hauptzählrichtung vorwärts [2]

3.1.2 Einmaliges Zählen bei der FM 350-2

Die Betriebsart „Einmaliges Zählen“ ermöglicht das Erfassen einer festen Anzahl von Zählimpulsen zwischen einem definierten Start- und Endwert. Über die Parametrierung werden Startwert, Endwert und Hauptzählrichtung festgelegt.

Beim Zählen in Hauptzählrichtung vorwärts beginnt der Zähler bei 0 und zählt einmalig bis zum Endwert. Wird der Wert „Endwert-1“ erreicht und ein weiterer Zählimpuls empfangen, springt der Zähler zurück auf den Zählerstand 0 und bleibt dort stehen, auch wenn weitere Impulse folgen.

Beim Zählen in Hauptzählrichtung rückwärts startet der Zähler beim Startwert und zählt einmalig in Richtung 0. Erreicht der Zähler den Wert 1 und ein weiterer Zählimpuls erfolgt, springt er auf den Startwert zurück und bleibt stehen.

Wird entgegen der gewählten Hauptzählrichtung gezählt und dabei der Startwert unter- oder überschritten, meldet die Baugruppe den aktuellen Zählerstand mit negativem Vorzeichen zurück. Ein Überlauf oder Unterlauf findet in diesem Modus nicht statt.

Ist ein Vergleichswert parametrierbar, kann bei aktuellem Zählerstand = Vergleichswert ein Prozessalarm ausgelöst und/oder ein Ausgang geschaltet werden.

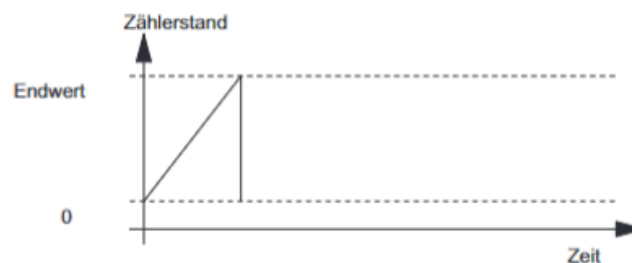


Abbildung 3.3: Einmaliges Zählen in Hauptzählrichtung vorwärts [2]

3.1.3 Periodisches Zählen bei der FM 350-2

Die Betriebsart „Periodisches Zählen“ erlaubt das Erfassen von Zählimpulsen innerhalb eines festgelegten Bereichs zwischen Startwert und Endwert. Nach Erreichen des Endwerts springt der Zähler automatisch auf den Startwert zurück und zählt erneut bis zum Endwert. Dieses Verhalten wiederholt sich zyklisch, wodurch eine periodische Zählung realisiert wird.

Beim Zählen in Hauptzählrichtung vorwärts startet der Zähler beim Startwert (meist 0) und zählt bis zum Endwert. Sobald der Endwert erreicht ist und ein weiterer Zählimpuls erfolgt, springt der Zähler zurück auf den Startwert und beginnt erneut zu zählen. Die Zählimpulse werden kontinuierlich erfasst, sodass der Zählvorgang periodisch erfolgt.

Beim Zählen in Hauptzählrichtung rückwärts startet der Zähler am parametrierten Startwert und zählt in Richtung Endwert (meist 0). Wird der Endwert erreicht und ein weiterer Zählimpuls empfangen, springt der Zähler auf den Startwert zurück und zählt erneut rückwärts.

Wird entgegen der gewählten Hauptzählrichtung gezählt und dabei der Startwert unter- oder überschritten, meldet die Baugruppe den aktuellen Zählerstand mit negativem Vorzeichen zurück. Ein Überlauf oder Unterlauf findet in diesem Modus nicht statt; der Ausgangswert bleibt unverändert.

Ist ein Vergleichswert parametriert, kann bei aktuellem Zählerstand = Vergleichswert ein Prozessalarm ausgelöst und/oder ein Ausgang geschaltet werden.

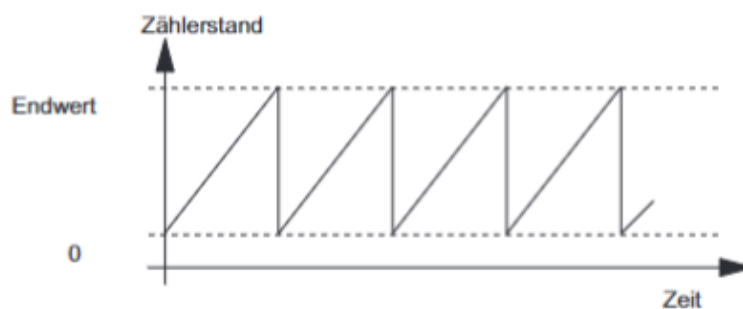


Abbildung 3.4: Periodisches Zählen in Hauptzählrichtung vorwärts [2]

3.1.4 Frequenzmessung mit der FM 350-2

Die FM 350-2 bietet eine integrierte Funktion zur Frequenzmessung, bei der die Anzahl der Impulse innerhalb eines parametrierbaren Zeitfensters gezählt wird. Die Zeitfensterlänge kann zwischen 10 ms und 10 s eingestellt werden. Nach Ablauf des Zeitfensters wird die Frequenz als Anzahl der gezählten Impulse pro Sekunde berechnet und als Wert in Hz ausgegeben.

Die Frequenzmessung kann sowohl über das Anwenderprogramm als auch über externe Signale (Hardware-Tor) gestartet und gestoppt werden. Die Baugruppe unterstützt dabei verschiedene Vergleichswerte, sodass Prozessalarme ausgelöst werden können, wenn die gemessene Frequenz bestimmte Grenzwerte über- oder unterschreitet.

Die wichtigsten Merkmale der Frequenzmessung sind:

- Messbereich: 0 Hz bis 9 999 999 Hz (abhängig von Parametrierung und Signalart)
- Einstellbare Zeitfenster für die Messung (10 ms bis 10 s)
- Prozessalarme bei Über- oder Unterschreiten von Grenzwerten
- Start und Ende der Messung über Software- oder Hardware-Tor
- Frequenzvergleich mit parametrierten Sollwerten möglich

Die Frequenzmessung eignet sich besonders für Anwendungen, bei denen die Geschwindigkeit oder Drehzahl von Maschinen und Anlagen überwacht werden muss. Typische Einsatzfälle sind die Überwachung von Förderbändern, Drehtischen oder Antrieben.

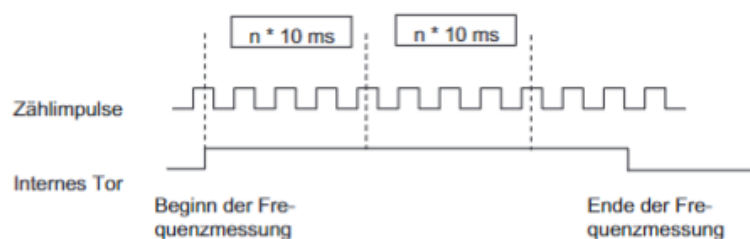


Abbildung 3.5: Prinzip der Frequenzmessung mit Zeitfenster bei der FM 350-2 [2]

3.1.5 Torfunktionen der FM 350-2

Zahlreiche Anwendungen erfordern, dass der Zählvorgang erst zu einem definierten Zeitpunkt, abhängig von anderen Ereignissen, gestartet oder gestoppt wird. Dieses Starten und Stoppen des Zählvorgangs geschieht bei der FM 350-2 über eine Torfunktion. Wird das Tor geöffnet, können Zählimpulse zum Zähler gelangen, der Zählvorgang wird fortgesetzt. Wird das Tor geschlossen, können keine Zählimpulse mehr zum Zähler gelangen, der Zählvorgang ist gestoppt.

Software-Tor und Hardware-Tor Die FM 350-2 besitzt zwei Torfunktionen:

- **Software-Tor (SW-Tor):** Das Software-Tor wird durch das Steuerbit `SW_GATE` im Anwenderprogramm gesetzt oder zurückgesetzt. Es kann ausschließlich durch einen Flankenwechsel von 0 auf 1 geöffnet und von 1 auf 0 geschlossen werden.
- **Hardware-Tor (HW-Tor):** Das Hardware-Tor wird durch ein externes Signal an einem digitalen Eingang der Baugruppe gesteuert. Es wird durch einen Flankenwechsel des Eingangssignals geöffnet oder geschlossen.

Internes Tor Das interne Tor ergibt sich als logische UND-Verknüpfung von HW-Tor und SW-Tor. Ist eines der Tore geschlossen, bleibt das interne Tor ebenfalls geschlossen und der Zählvorgang ist inaktiv. Nur wenn beide Tore offen sind, ist das interne Tor aktiv und der Zählvorgang läuft.

HW-Tor	SW-Tor	internes Tor	Zählvorgang
offen	offen	offen	aktiv
offen	geschlossen	geschlossen	inaktiv
geschlossen	offen	geschlossen	inaktiv
geschlossen	geschlossen	geschlossen	inaktiv

Tabelle 3.2: Zusammenhang der Torfunktionen und des Zählvorgangs [2]

Beispiel Mit dem Setzen des Torsignals wird das Tor geöffnet und die Zählimpulse werden gezählt. Wird das Torsignal weggenommen, wird das Tor geschlossen und die Zählimpulse werden nicht mehr vom Zähler erfasst. Der Zählerstand bleibt konstant.

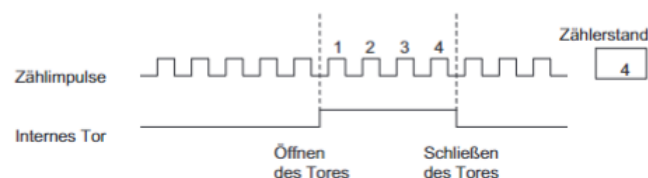


Abbildung 3.6: Öffnen und Schließen eines Tores und das Zählen der Impulse [2]

3.1.6 Prozessalarme der FM 350-2

Im Betrieb der Zählerbaugruppe FM 350-2 können verschiedene Fehler auftreten. Diese entstehen z. B. durch Defekte auf der Baugruppe, fehlerhafte Bedienung/-Verdrahtung oder widersprüchliche Parametrierung. Zur Erkennung und gezielten Reaktion unterscheidet die Baugruppe Fehlerklassen und löst bei Bedarf Prozessalarme aus.

Fehlerklassen und -arten

- Interner Fehler: Defekte oder fehlerhafte Zustände innerhalb der Baugruppe, z. B. Ansprechen der Zeitüberwachung (Watchdog).
- Externer Fehler: Probleme in der Peripherie außerhalb der Baugruppe, die keinem einzelnen Kanal zugeordnet werden können.
- Externer Kanalfehler: Kanalspezifische Fehler, z. B. Signalfehler durch einen NAMUR-Geber.
- Datenfehler: Unzulässige Aufträge von Automatisierungsgerät (AG) oder Programmiergerät (PG), z. B. Vergleichswerte außerhalb des Zählbereichs.

Reaktion der Baugruppe

- Interne, externe und externe Kanalfehler: Sammelfehler-LED (SF) leuchtet; es können Diagnosealarme ausgelöst werden (wenn freigegeben).
- Datenfehler: Auftrag wird abgelehnt; Eintrag im Diagnosepuffer; Behebung durch erneute Übertragung mit korrigierten Daten.

Sammelfehler-LED (SF) Leuchtet rot bei internem Baugruppenfehler, fehlerhafter Parametrierung oder Leitungsfehlern. Typische Ursachen: angesprochene Zeitüberwachung, verlorener Prozessalarm, fehlende/fehlerhafte Parametrierung, Probleme der Geberversorgung oder fehlerhafte NAMUR-Gebersignale (z. B. Drahtbruch/Kurzschluss).

Diagnosealarme und -daten Diagnosealarme können in der Parametrierung freigegeben werden. Beim Fehler ruft die CPU den Diagnose-OB OB 82 auf; das zyklische Programm wird unterbrochen.

- Diagnosedaten DS0: werden beim Aufruf von OB 82 automatisch übertragen.
- Diagnosedaten DS1: detaillierte Kanalinfos, auslesbar über FC DIAG_RD.
- Inhalt: Hinweise zu intern/extern/Parametrier-/Kanalfehlern, Ansprechen der Zeitüberwachung, verlorene Prozessalarme, Leitungs-/Geberversorgungsfehler.
- Zusätzlich können über SFC 52 benutzerdefinierte Meldungen in den Diagnosepuffer der CPU geschrieben werden.

Datenfehler und Quittierung Ergeben sich aus unzulässigen Aufträgen durch PG oder über FC CNT2_WR bzw. FB CNT2WRPN. Die Baugruppe prüft und lehnt fehlerhafte Aufträge ab; das Bit DATA_ERR im Zähler-Datenbaustein wird gesetzt. Behebung durch Korrektur und erneute Übertragung des Auftrags. Anzeige im Diagnosepuffer sowie im Menü Test > Fehlerauswertung.

3.2 Eigenschaften und Aufbau des TM FAST

Das TM FAST ermöglicht die Integration eigener Anwendungen in das SIMATIC S7-1500 Technologiemodul. Hardwareseitig besteht es aus einem Backend-Baustein, der die Siemens TM FAST Firmware bereitstellt, und einem Frontend-Baustein, dem Intel® Cyclone 10 LP FPGA, auf dem die TM FAST Anwendung läuft. Diese Anwendung definiert die Hardwarefunktionalität des Moduls und kann durch applikationsspezifische FPGA-Programmierung mittels Intel® Quartus® Prime angepasst werden [11, 3].

Eine ähnliche Baugruppe ist der High Speed Boolean Processor FM352-5 des SIMATIC S7-300 Systems, der mit einer Zykluszeit von 1 μ s schnelle Binärsteuerungen ermöglicht [19]. Das TM FAST FPGA bietet jedoch eine wesentlich höhere Flexibilität, erlaubt Zykluszeiten bis 20 ns und die Erzeugung präziser Pulsmuster. Somit können nicht nur digitale Signale, sondern auch komplexere Aufgaben wie Zählvorgänge, präzise Dosierungen oder pixelgenaue Punktmuster verarbeitet werden [3].

- Positionsabhängige Nocken
- Schnelle Reaktionen auf digitale Ereignisse
- Zählen von Signalen bis 5 MHz
- Auswertung von Inkremental- und Absolutwertgebern mit sofortiger Reaktion
- Präzise Steuerung von Lasern
- Berechnungen und Vorverarbeitungen im Modul (z. B. Fehlerfrüherkennung)

Die freie Programmierbarkeit ermöglicht die Kombination und Anpassung einzelner Funktionen. Das TM FAST eignet sich daher besonders für Anwendungen, bei denen die SPS mit Peripheriemodulen nicht ausreichend schnell ist oder spezifische Einzelanwendungen erfordert.

Aufbau des TM FAST

Das TM FAST besteht aus einem Backend-Gerät, einem Frontend-Gerät, einer Prozessschnittstelle sowie der Anbindung an CPU und Peripherie (vgl. Abb. 3.7).

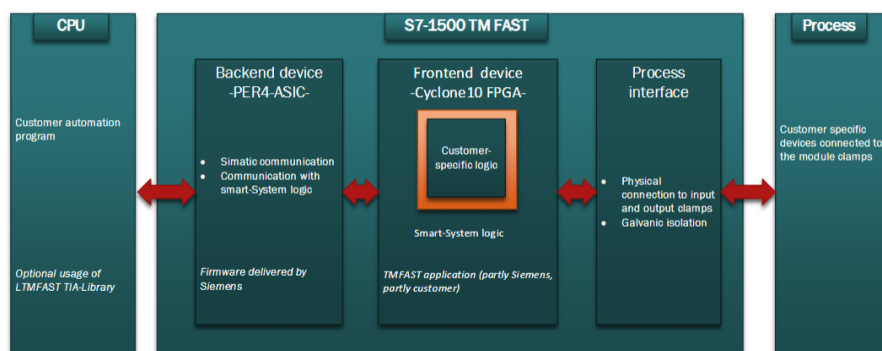


Abbildung 3.7: Blockdiagramm des TM FAST [11]

Backend-Gerät: Das Backend sorgt für die Kommunikation des TM FAST mit der S7-1500 über den Rückwandbus, regelt zyklischen und azyklischen Datentransfer, überwacht den Diagnosestatus und lädt die TM FAST-Anwendung auf das FPGA. Zudem kontrolliert es die Systemparameter, die Modultemperatur und die Statusanzeigen [3].

Frontend-Gerät: Im Frontend läuft die anwendungsspezifische TM FAST Software auf dem Intel® Cyclone 10 LP FPGA. Die Logik ist in zwei Teile unterteilt: FAST-Systemlogik (TFL FAST SYS) und Benutzerlogik (TFL FAST USER). Die Programmierung erfolgt in VHDL über Top-Entity-Strukturen, Architekturen und Bibliotheken. Über diese Schnittstellen können Ein- und Ausgänge gesteuert sowie die Kommunikation mit der CPU realisiert werden (Abb. 3.8).

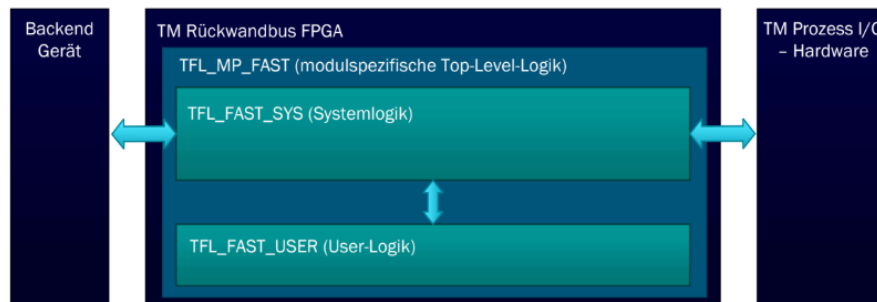


Abbildung 3.8: Aufbau von Frontend- und Backend-Gerät und Einbindung in das System [11]

Das FPGA-Modell 10CL025YU256A7G verfügt über 24 624 Logikelemente, 66 M9K Memory-Blöcke, 66 18×18-Bit-Multiplizierer, vier PLLs, 20 Clock-Leitungen und maximal 150 I/Os (Tab. 3.3) [10]. Jeder Memory-Block bietet 9 Kbit Speicher, die als RAM, FIFO oder ROM verwendet werden können. Die PLLs sind dynamisch rekonfigurierbar, und die GPIOs bieten Pull-Up-Widerstände und programmierbare Anstiegszeiten.

Ressource	Anzahl
Logikelemente	24 624
M9K Memory Blöcke	66
18×18 Multiplizierer	66
PLL	4
Clocks	20
Maximale I/O	150

Tabelle 3.3: Ressourcen des Intel® Cyclone 10 LP FPGA [10]

Äußerer Aufbau: Das Modul ist als Baugruppe der Bauform MP mit einem 40-poligen Frontstecker und vier zusätzlichen Klemmen zur Spannungsversorgung

ausgeführt (Abb. 3.9). Ein TM FAST Debug Connector ermöglicht Service- und Entwicklungszugriffe unabhängig vom Frontstecker [3].

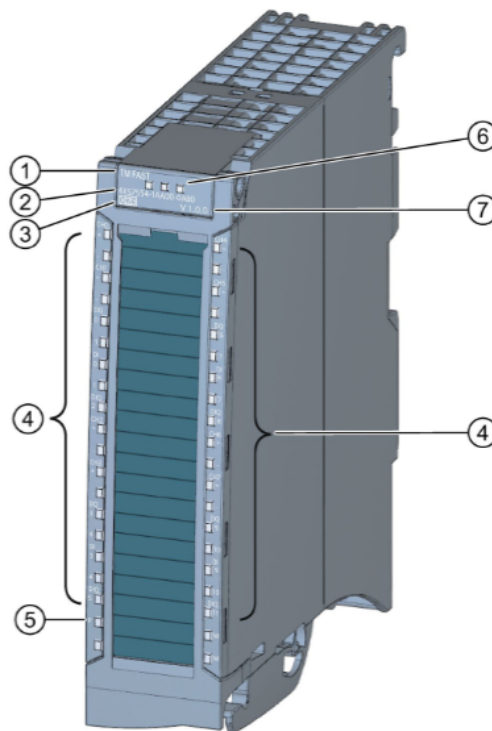


Abbildung 3.9: Technologiemodul TM FAST [3]

Das Modul bietet je acht 24 V-Digitaleingänge (DI) und -ausgänge (DQ) mit 0,1 A Ausgangsstrom sowie vier DIQ-Kanäle mit 0,3 A und maximal 1 kHz. Zudem stehen acht RS422/485-Kanäle bereit, die auch als TTL-Signale verwendet werden können. Eingangsverzögerungen für Digitaleingänge und RS422/485/TTL-Kanäle lassen sich sowohl fest parametrieren als auch über die TM FAST Anwendung programmieren.

Kommunikation: Die Datenübertragung zwischen CPU und TM FAST kann zyklisch oder azyklisch erfolgen. Zyklische Ein- und Ausgangsdaten bestehen aus acht Doppelwörtern (32 Bit je Doppelwort). Die CPU-Schnittstelle betrachtet die Steuer- (CTRL IF) und Rückmeldedaten (FB IF). Azyklische Datenmengen können je nach Anwendungslogik konfiguriert werden (4, 32, 64 oder 128 Byte) und über die TIA Portal-Befehle `LTMFAST UserWriteREC` bzw. `LTMFAST UserReadREC` gelesen bzw. geschrieben werden [11, 3].

3.3 Direkter Vergleich der TM FAST und FM 350-2

Kriterium	TM FAST	FM 350-2
Unterschiede		
Architektur	Intel Cyclone 10 LP FPGA, frei programmierbar (VHDL)	Feste ASIC-basierte Zählerbaugruppe, keine freie Hardwareprogrammierung
Zählerkanäle	Bis zu 14 Kanäle, flexibel konfigurierbar	8 Kanäle, fest als Zähler ausgelegt
Geschwindigkeit	Eingangssignale bis zu 5 MHz (RS422/485/DIQ)	Max. 20 kHz (A0...7), 10 kHz (B0...7)
Zykluszeit	20 ns FPGA-Takt (hohe Parallelität)	50 μ s typische Verarbeitungszeit
Betriebsarten	Frei programmierbar (z.B. Dosierlogik, Pulsfolgen)	Vorgegebene Betriebsarten (Endlos, periodisch, Dosieren etc.)
Parametrierung	Änderungen im laufenden Betrieb über UserWriteREC	Parametrierung nur über STEP 7, meist im Stopp-Betrieb
Integration	S7-1500, TIA Portal	S7-300, STEP 7
Gemeinsamkeiten		
Zählfunktionen	Hohe Zählauflösung (32 Bit), schnelle Verarbeitung, Einsatz als Zähler in der Automatisierungstechnik	
Industrielle Anwendungen	Verpackung, Sortierung, Dosierung, Maschinenbau, Prozessautomatisierung	

Tabelle 3.4: Kompakter Vergleich der Eigenschaften von TM FAST und FM 350-2 (Daten gemäß [3, 9, 2]).

Im Vergleich zeigt sich, dass das TM FAST Modul nicht nur mehr Kanäle bietet, sondern auch eine deutlich höhere Signalgeschwindigkeit und Flexibilität in der Programmierung. Durch die FPGA-Architektur wird zudem eine extrem kurze Zykluszeit von nur 20 ns erreicht, während das FM 350-2 mit typischen 50 μ s deutlich langsamer arbeitet. Damit eignet sich das TM FAST insbesondere für moderne, hochkanalige Anwendungen, bei denen sich Anforderungen dynamisch ändern können. Das FM 350-2 ist dagegen fest verdrahtet und limitierter, war aber

für klassische Zähler- und Dosieraufgaben in der S7-300-Umgebung lange Zeit eine etablierte Lösung.

Ein wesentlicher Vorteil des FM 350-2 liegt in der direkten Unterstützung von A- und B-Signalen pro Kanal für den Betrieb von Inkrementalgebern. Diese Funktionalität ist beim TM FAST prinzipiell ebenfalls umsetzbar, erfordert jedoch eine Mehrfachbelegung einzelner Eingänge, was die effektiv nutzbare Kanalanzahl halbieren würde.

4 Entwicklung des VHDL-basierten Applikationsbeispiels

In diesem Kapitel wird die Entwicklung des VHDL-basierten Applikationsbeispiels für das TM FAST beschrieben. Ausgehend von den funktionalen und technischen Anforderungen wird die Architektur des Moduls vorgestellt, wobei Aspekte wie Modularisierung, Ressourcenoptimierung, Fehlerbehandlung und Kommunikationsstrategien im Vordergrund stehen. Darüber hinaus werden Mechanismen zur sicheren Betriebsführung, flexible Kanaluordnungen sowie Strategien zur Überlaufkompensation erläutert.

4.1 Anforderungen an die Neuentwicklung

Die Arbeit verfolgt insbesondere folgende Ziele:

- Umsetzung der bekannten FM350-2-Funktionalitäten für Dosieranwendungen auf dem TM FAST Modul.
- Erhöhung der Kanalanzahl von 8 auf die höchstmögliche Anzahl für flexible Dosieranwendungen.
- Optimierung der Implementierung hinsichtlich Performance und Ressourcenschonung.
- Integration des Applikationsbeispiels in das TIA Portal zur nahtlosen Nutzung in industriellen Automatisierungslösungen.
- Anwendung bewährter Methoden des Software Engineerings, um Codequalität und Skalierbarkeit sicherzustellen.

Daraus bilden sich folgenden funktionale und nichtfunktionale Anforderungen:

4.1.1 Funktionale Anforderungen

- F1 Das Modul stellt mindestens acht unabhängige Zählerkanäle bereit.
- F2 Jeder Kanal kann flexibel als Digitaleingang, Digitalausgang oder RS485/TTL-Schnittstelle verwendet werden.
- F3 Jeder Kanal verwendet eine 32-Bit-Zählvariable.
- F4 Die Übergabe von Sollwerten und Parametern ist im laufenden Betrieb möglich.
- F5 Das Modul unterstützt die Betriebsmodi Rundenzähler, Reset, Hold und Start/Stop.
- F6 Für jeden Kanal werden Statusinformationen und Prozesswerte rückgemeldet.
- F7 Fehler werden erkannt, sicher behandelt und an die Steuerung gemeldet.

4.1.2 Nichtfunktionale Anforderungen

- N1 Die Lösung ist vollständig in das TIA Portal integrierbar.
- N2 Die Funktionalität entspricht mindestens dem Umfang der FM350-2, der für die Umsetzung einer Dosieranwendung erforderlich ist.
- N3 Ressourcenschonende Implementierung (FPGA-Auslastung $< 70\%$ bei 14 Kanälen).
- N4 Verbesserte Reaktionszeit (FPGA-Reaktionszeit $< 50 \mu\text{s}$).
- N5 Erweiterbarkeit auf zusätzliche Kanäle ohne Anpassung der Grundarchitektur.

4.2 Umsetzung der funktionalen Anforderungen

In diesem Abschnitt werden die zuvor spezifizierten funktionalen Anforderungen systematisch umgesetzt und im Detail beschrieben. Ziel ist es, die theoretisch definierten Anforderungen in ein praxisnahes und nachvollziehbares VHDL-Design zu überführen. Dabei werden die einzelnen Funktionen nicht nur auf ihre technische Realisierung hin erläutert, sondern auch in Bezug auf ihre Relevanz für den Gesamtkontext der Dosieranwendung eingeordnet.

F1 – mindestens Acht unabhängige Zählerkanäle und Modularisierung

Die Umsetzung erfolgt durch den Einsatz von Arrays und **generate**-Blöcken in VHDL, wodurch jeder Kanal logisch getrennt und unabhängig voneinander betrieben werden kann. Dadurch ist die Anzahl der Kanäle nur durch die physischen Eingänge und die Anzahl der verfügbaren Logikelemente begrenzt.

Listing 4.1: Generate-Struktur für mehrere Kanäle

```

1 channel_gen : for i in 0 to CHANNEL_COUNT-1 generate
2     — Kanal-Logik
3 end generate;
```

Darüber hinaus ist die Architektur des VHDL-Designs konsequent modular aufgebaut und nutzt Generics, Arrays und Generate-Statements, um eine hohe Wiederverwendbarkeit und Erweiterbarkeit zu gewährleisten.

So wird z.B. die Anzahl der Kanäle über das Generic `CHANNEL_COUNT` zentral festgelegt. Für alle kanalbezogenen Signale und Zustände werden Array-Typen verwendet, sodass die Logik für jeden Kanal identisch und unabhängig voneinander instanziiert werden kann.

Durch die Verwendung von Generate-Blöcken (`channel_gen : for i in 0 to CHANNEL_COUNT-1 generate`) wird die komplette Zähler- und Frequenzmesslogik pro Kanal automatisch erzeugt. Neue Kanäle können einfach durch Erhöhen von `CHANNEL_COUNT` und Anpassen der Mapping-Arrays hinzugefügt werden, ohne dass das Gesamtdesign oder die Prozesslogik angepasst werden muss.

F2 – Flexible Kanalnutzung und Zuordnung

Die physikalischen Schnittstellen des TM FAST erlauben eine flexible Nutzung der Kanäle als Digitaleingänge, Digitalausgänge oder RS485/TTL-Schnittstellen. Die logische Kanalzuordnung erfolgt über Mapping-Arrays, wodurch die Kanalbelegung ohne Änderung der eigentlichen Zählerlogik an verschiedene Hardwarekonfigurationen angepasst werden kann.

Listing 4.2: Beispiel einer Kanalzuordnung

```

1 type t_channel_map is array (0 to 13) of integer;
2 constant CHANNEL_MAP_IN_CH  : t_channel_map
3 := (0, 1, 3, 4, 6, 7, 9, 10, 0, 1, 2, 3, 4, 5);
4 constant CHANNEL_MAP_OUT_CH : t_channel_map
5 := (0, 1, 3, 4, 6, 7, 9, 10, 2, 5, 8, 11, 6, 7);

```

Im Beispiel 4.2 sind die ersten acht Kanäle als klassische DI/DQ-Kanäle definiert. Die übrigen Eingänge werden flexibel als RS485 verwendet, während die Ausgänge auf DIQ- sowie zusätzliche RS485-Kanäle verteilt sind. Über die Arrays CHANNEL_MAP_IN_CH und CHANNEL_MAP_OUT_CH kann die Belegung zentral geändert werden, ohne dass die Prozesslogik angepasst werden muss.

Ein praktischer Vorteil für Dosieranwendungen ist, dass bei Inkrementalgebern häufig nur die A-Flanke ausgewertet werden muss, da die Förderrichtung fix ist. Dadurch genügt pro Kanal ein Eingang und ein Ausgang, sodass sich alle 28 physischen Schnittstellen ([3]) optimal auf 14 logische Kanäle aufteilen lassen.

Diese Architektur erhöht nicht nur die Flexibilität, sondern auch die Wiederverwendbarkeit und Wartbarkeit des Designs: Änderungen der Verdrahtung oder neue Anforderungen können allein durch Anpassung der Mapping-Arrays umgesetzt werden.

F3 – 32-Bit-Zählvariable

Jeder Kanal verwendet eine `std_logic_vector(31 downto 0)`, um ausreichend Zählbereich für industrielle Anwendungen sicherzustellen.

F4 – Übergabe von Sollwerten im laufenden Betrieb

Für die Übergabe von Sollwerten und Parametern im laufenden Betrieb wird die azyklische Schnittstelle `WR_REC` verwendet. Eine rein zyklische Übertragung über `CTRL_IF` wäre auf acht Doppelwörter beschränkt und würde nicht für 14 Kanäle ausreichen. Mit `WR_REC` stehen dagegen 64 Byte (16 Doppelwörter) zur Verfügung, was alle 14 Kanäle mit je 32 Bit Sollwert abdeckt und noch Platz für zusätzliche Steuerbits lässt.

Kommunikationsaufteilung: Das Design trennt klar zwischen zyklischer und azyklischer Kommunikation: - *Zyklische Steuerung* (`CTRL_IF`, `FB_IF`) überträgt zeitkritische Steuer- und Statusinformationen in jedem SPS-Zyklus. Dazu gehören z. B. `CounterReset`, `EnableDQ`, `Load`, `Hold`, `StartDQ`, `Polarity` und die Auswahl der Rückmeldedaten (`FBIF_Control`). - *Azyklische Sollwertübertragung* (`WR_REC`, `RD_REC`) erlaubt das Setzen neuer Obergrenzen (Sollwerte) oder globaler Steuerparameter (`WR_REC_Control`) während des Betriebs. Änderungen werden zunächst gepuffert und erst durch ein zyklisches Steuersignal aktiviert.

```

1  if WR_REC_NEW = '1' then
2      next_upper_limit(i) <= WR_RECxx;
3  end if;
4
5  if Load(i) = '1' then
6      upper_limit(i) <= next_upper_limit(i);
7  end if;

```

Listing 4.3: Übernahme azyklischer Sollwerte in den Puffer

Synchronisation und Zustandsautomat: Um Race Conditions zu vermeiden, steuert ein einfacher Zustandsautomat die Sollwertübernahme: 1. Bei einer Flanke auf `WR_REC_NEW` wird der neue Sollwert in `next_upper_limit(i)` gespeichert. 2. Erst wenn `Load(i)` aktiv wird, übernimmt der Kanal den Wert in `upper_limit(i)`. 3. Gleichzeitig wird geprüft, ob das Limit bereits erreicht wurde, und ggf. `LimitReached(i)` gesetzt. 4. Nach Loslassen von `Load(i)` geht der Automat in den Grundzustand zurück.

```

1  case load_delay_cnt(i) is
2      when 0 =>
3          if Load(i) = '1' then
4              upper_limit(i) <= next_upper_limit(i);
5              load_delay_cnt(i) <= 1;

```

```

6         if unsigned(enc_count(i)) >= unsigned(next_upper_limit(i)) then
7             LimitReached(i) <= '1';
8         else
9             LimitReached(i) <= '0';
10        end if;
11    end if;
12    when 1 =>
13        if Load(i) = '0' then
14            load_delay_cnt(i) <= 0;
15        end if;
16    when others =>
17        load_delay_cnt(i) <= 0;
18 end case;

```

Listing 4.4: Zustandsautomat zur synchronisierten Sollwertübernahme

Dieses Zusammenspiel von zyklischer Steuerung und azyklischer Sollwertübertragung stellt sicher, dass Änderungen nur gezielt und deterministisch wirksam werden. Das System bleibt flexibel für Sollwertänderungen im Betrieb und gleichzeitig robust gegen undefinierte Zustände oder Wettlaufschaltungen.

F5 – Unterstützte Betriebsmodi und Überlaufkompensation

Für Dosieranwendungen ist der zentrale Betriebsmodus das *periodische Zählen*. Andere Modi wie *einmaliges Zählen* (benötigt zusätzliche Logik zur Überlaufkorrektur) oder *endloses Zählen* (keine Sollwerte) erfüllen die Anforderungen nicht.

Notwendigkeit der Überlaufkompensation: Beim Dosieren treten stets Verzögerungen durch mechanische Trägheit auf, etwa durch die Ventil-Schließzeit. Dadurch wird mehr Material als geplant eingefüllt (Nachlauf). Ohne Kompensation würden systematische Abweichungen entstehen [14]. Zur Korrektur existieren verschiedene Ansätze:

1. Prädiktion des Abschaltzeitpunkts (basierend auf Durchflussrate und Ventil-Schließzeit),
2. zweistufige Dosierung (grob + fein),
3. PID-geregelte Dosierung,

4. direkte Messung über Massendurchflussmesser.

Implementierte Lösung: automatische Überlaufkompensation Im FPGA-Design wird eine ressourcenschonende, adaptive Überlaufkompensation umgesetzt. Jeder Kanal besitzt einen 32-Bit-Zähler. Beim Start einer Dosierung wird das Sollwert-Limit geladen. Wird es erreicht, setzt das Signal `LimitReached` den Ausgang zurück. Der Nachlauf, der dennoch gezählt wird, geht in die Berechnung des nächsten Zyklus ein.

Die Logik lässt sich wie folgt beschreiben:

$$V_{\text{total}} = V_{\text{target}} + V_{\text{current_overflow}}$$

für die erste Abfüllung, und für nachfolgende Zyklen:

$$V_{\text{total}} = V_{\text{target}} + V_{\text{current_overflow}} - V_{\text{last_overflow}}$$

Unter stabilen Bedingungen ($V_{\text{current_overflow}} = V_{\text{last_overflow}}$) ergibt sich eine konstante Genauigkeit, da der Ventilynachlauf automatisch berücksichtigt wird.

Vor- und Nachteile: Die Methode kompensiert zuverlässig die Systemträgheit, erfordert keine zusätzliche Hardware, ist adaptiv, da der jeweils letzte Überlaufwert verwendet wird, und eignet sich besonders für kontinuierliche Prozesse mit konstanten Bedingungen.

Grenzen treten auf bei stark schwankenden Parametern (Druck, Temperatur, Viskosität) oder bei Produktwechseln, da das System nachgelernt werden muss. Auch bei sehr unterschiedlichen Füllmengen können mehrere Überlaufwerte erforderlich sein, um präzise zu bleiben.

Damit stellt die Überlaufkompensation eine robuste und effiziente Lösung dar, die die Genauigkeit in typischen Dosierprozessen deutlich verbessert.

F6 – Rückmeldungen

Das Feedback-Interface (FB_IF) liefert in jedem SPS-Zyklus aktuelle Status- und Prozesswerte für alle Kanäle. Jeder Kanal stellt ein 16-Bit-Wort bereit. Die Bitstruktur ist wie folgt:

- Bit 0–14: aktueller Prozesswert (z. B. Zählerstand oder Frequenz),
- Bit 15: Spiegelung des `Select_Feedback_Value`-Flags (Diagnose/Rückmeldung),
- Bit 16 (MSB): Status des Ventilausgangs (1 = Ausgang aktiv, 0 = Ausgang inaktiv).

Zusätzlich werden über das FB_IF-Interface Fehler- und Statussignale übertragen:

- `ErrorChannelNumber`: Nummer des letzten Kanals mit Fehler (0 = kein Fehler),
- `DI_DQ_BAD`: Bit-codierter Fehlerstatus für Digitaleingänge und -ausgänge,
- `RS485_LP_BAD`: Bit-codierter Fehlerstatus für RS485 oder L+,
- `StatusCode`: Applikations-Fehlercode und Aktivitätsanzeige.

Durch diese Rückmeldungen ist eine kontinuierliche Überwachung und Diagnose aller Kanäle durch die SPS möglich.

F7 – Fehlerbehandlung

Fehlerfälle werden durch gezielte Rücksetzung aller Signale behandelt. Ein Beispiel ist das Verhalten bei `CPU_STOP`.

Fehler- und Diagnosestrategien: Das VHDL-Design implementiert eine systematische Fehlererkennung und sichere Fehlerbehandlung auf Kanalebene. Für jeden Kanal wird das Signal `system_fault(i)` berechnet, das verschiedene Fehlerquellen überwacht:

- `DI_QI_BAD`: Fehler am zugeordneten Digitaleingang (z. B. Drahtbruch, Kurzschluss),

- DQ_QI_BAD: Fehler am zugeordneten Digitalausgang,
- RS485_QI_BAD: Fehler an den RS485-Ein- und Ausgängen (z. B. Kommunikationsfehler, Pegelstörung),
- LP_QI_BAD: Fehler in der Spannungsversorgung (Low Power).

Die Fehler werden im Prozess `fault_dq_proc` für jeden Kanal individuell ausgewertet. Sobald ein Fehler erkannt wird (`system_fault(i) = '1'`), wird der entsprechende Ausgang (DQ oder RS485) automatisch in einen definierten Zustand (SAFE State) überführt. Der SAFE State ist abhängig von der konfigurierten Polarität und verhindert unkontrollierte Aktionen im Fehlerfall.

Die Fehlerdetails werden über das Feedback-Interface (z. B. FB_IF7) an die SPS rückgemeldet. Hierbei werden die Fehlerbits für DI, DQ, RS485 und die Kanalnummer gesetzt, sodass die Steuerung gezielt auf den Fehler reagieren kann. Ist kein Fehler vorhanden, werden die Fehlerbits explizit gelöscht.

Zusätzlich wird im Fehlerfall der Ausgang sofort deaktiviert und der Status `StartDQ_active` zurückgesetzt, um einen sicheren Anlagenzustand zu gewährleisten. Im Fall eines globalen CPU-Stop (`CPU_STOP = '1'` und `WR_REC_Control = x"00000000"`) werden alle Ausgänge in den SAFE State gesetzt und alle Statusregister zurückgesetzt. Dieses Konzept stellt sicher, dass sowohl Einzelkanalfehler als auch globale Fehler jederzeit erkannt und sicher behandelt werden. Die SPS erhält über die Rückmeldebite eine detaillierte Diagnose und kann gezielt Maßnahmen einleiten.

Sicherheit bei CPU_STOP: Im Fehlerfall oder bei einem CPU_STOP muss das System in einen definierten Zustand übergehen, um unkontrollierte Aktionen und Gefährdungen zu verhindern. Dies wird durch gezielte Abfragen und Rücksetzungen aller relevanten Steuer- und Ausgangssignale erreicht.

- Bei `WR_REC_Control = x"00000000"` werden alle Ausgänge sofort ausgeschaltet und alle Werte aller Kanäle zurückgesetzt (Abort Mode).
- Bei `WR_REC_Control = x"00000001"` läuft der Continue Mode. Die Kanäle setzen ihre aktuelle Operation fort und gehen nach deren Ende in einen definierten Zustand über, aus dem die Operation bei einem CPU-Start wieder aufgenommen werden kann.

Ein Auszug aus dem Code illustriert das Verhalten:

```

1 StartDQ(i) <= '0'; -- always 0 when CPU-STOP
2 if WR_REC_Control = x"00000000" then
3     -- Abort mode
4     CounterReset(i) <= '1';
5     polarity(i) <= '0';
6     EnableDQ(i) <= '0';
7     Load(i) <= '0';
8     HoldSW(i) <= '0';
9     FBIF_Control(i) <= '0';
10 else
11     -- Continue mode
12     if StartDQ_active(i) = '1' then
13         EnableDQ(i) <= '1';
14     end if;
15 end if;

```

Listing 4.5: Rücksetzen aller Ausgänge und Register im CPU_STOP-Fall

Durch diese Maßnahmen wird sichergestellt, dass das System im Fehlerfall oder bei einem CPU_STOP deterministisch und zuverlässig in einen definierten Zustand übergeht. Dabei handelt es sich jedoch nicht um eine SIL-Klassifizierung oder funktionale Sicherheit im engeren Sinne, sondern um die Gewährleistung eines stabilen Prozesses mit klar definierten Zuständen. Unerwünschte Schalthandlungen oder undefinierte Zustände werden verhindert. Für funktionale Sicherheit im industriellen Maßstab sind spezielle F-CPU's und Safety-Module vorgesehen, wie sie im Rahmen von SIMATIC Safety Integrated angeboten werden [24].

4.3 Umsetzung der nicht-funktionalen Anforderungen

In diesem Abschnitt werden die zuvor definierten nicht-funktionalen Anforderungen systematisch berücksichtigt und in das Gesamtdesign integriert. Der Fokus liegt hierbei weniger auf der konkreten Abbildung einzelner Funktionen, sondern auf der Gewährleistung übergreifender Qualitätsmerkmale wie Performance, Ressourcenschonung, Erweiterbarkeit und Funktionsumfang.

Neben der technischen Realisierung werden die Maßnahmen jeweils in Bezug auf ihre Bedeutung für die Gesamtanwendung eingeordnet. Dadurch wird deutlich,

inwiefern die nicht-funktionalen Anforderungen die Leistungsfähigkeit, Stabilität und langfristige Nutzbarkeit der Lösung im industriellen Kontext sicherstellen.

N1 – Integration in das TIA Portal

Die Einbindung der entwickelten Applikationsbausteine in das TIA Portal ist ein zentraler Schritt, um die Funktionalität praxisnah nutzbar zu machen. Dazu werden Funktionsbausteine erstellt, Schnittstellen mit dem TM FAST definiert und die Anbindung an die Prozessabbildung realisiert.

Ein wesentlicher Baustein ist `LTMFAST_ControlREC`, der die azyklische Steuerung übernimmt. Er erhält die notwendigen Informationen von den Funktionsbausteinen `UserReadREC` und `UserWriteREC`. Darüber werden Zustände wie „Kommunikation gestartet“, „Kommunikation läuft“, „Kommunikation erfolgreich abgeschlossen“ oder „Fehler aufgetreten“ sowie die zu übertragenden Nutzdaten abgebildet. Für jeden Funktionsbaustein muss eine Instanz angelegt und im Projekt mitgeführt werden (vgl. Listing 4.6).

```

1 #LTMFAST_UserWriteREC_Instance(
2     req           := #loadrequest ,
3     busy          => #Busy ,
4     done          => #Done ,
5     error         => #Error ,
6     status        => #Status ,
7     userDataContent := #userDataContent ,
8     TMFASTControlREC :=
9         "TestHandler_LTMFAST_ControlREC_DB " .
10         LTMFAST_ControlREC_Instance
11 );

```

Listing 4.6: Funktionsaufruf von `#LTMFAST_UserWriteREC_Instance`

Für die Steuerung des TM FAST dient der Baustein `LTMFAST_dosing_application`. Er bildet die Schnittstelle zum TM FAST Modul und setzt die Eingangssignale in das vorgesehene Bitmuster um (vgl. Listing 4.7). Die Signale werden über die I/O-Schnittstelle übertragen und mit den Steuerbits `Hold`, `Load`, `CounterReset`, `StartDQ`, `Select_Feedback_Value`, `polarity` und `EnableDQ` umgesetzt.

```

1 FOR #ChannelIdx := 1 TO "Number_of_activ_Channels" DO
2
3     #AktChannel := #Channels[#ChannelIdx];
4     #Temp_CTRL_Bits[#ChannelIdx] := WORD#16#0000;
5     #LoadRequestArray[#ChannelIdx] := FALSE;
6
7     IF #AktChannel.Hold = TRUE THEN
8         #Temp_CTRL_Bits[#ChannelIdx] := WORD#16#1000;
9     ELSIF #AktChannel.CounterReset = TRUE THEN
10        #Temp_CTRL_Bits[#ChannelIdx] := WORD#16#0100;
11    ELSIF #AktChannel.Load = TRUE THEN
12        #Temp_CTRL_Bits[#ChannelIdx] := WORD#16#0800;
13        #LoadRequestArray[#ChannelIdx] := TRUE;
14    ELSIF #AktChannel.StartDQ = TRUE THEN
15        #Temp_CTRL_Bits[#ChannelIdx] := WORD#16#2000;
16    END_IF;
17
18    IF #AktChannel.EnableDQ = TRUE THEN
19        #Temp_CTRL_Bits[#ChannelIdx] :=
20        #Temp_CTRL_Bits[#ChannelIdx] OR WORD#16#0400;
21    END_IF;
22
23    IF #AktChannel.polarity = TRUE THEN
24        #Temp_CTRL_Bits[#ChannelIdx] :=
25        #Temp_CTRL_Bits[#ChannelIdx] OR WORD#16#0200;
26    END_IF;
27
28    IF #AktChannel.Select_Feedback_Value = TRUE THEN
29        #Temp_CTRL_Bits[#ChannelIdx] :=
30        #Temp_CTRL_Bits[#ChannelIdx] OR WORD#16#4000;
31    END_IF;
32
33 END_FOR;

```

Listing 4.7: Generierung der Steuerbits für die zyklische Kommunikation

Wie in Abbildung 4.1 gezeigt, wird der Funktionsbaustein direkt im OB1 eingebunden und über den selbst definierten Datentyp (UDT) **Channels** angesteuert. Dieser Datentyp enthält alle für die zyklische Kommunikation relevanten Steuerungen.

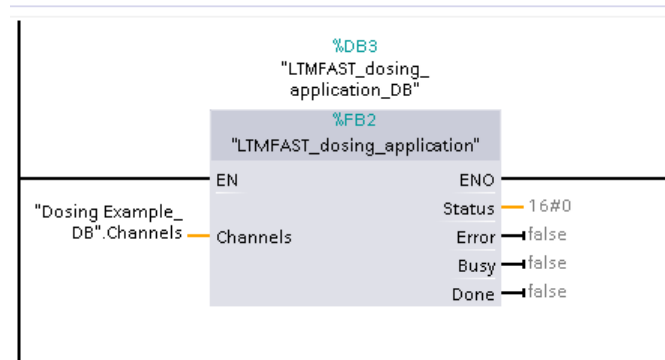


Abbildung 4.1: Integration des Applikationsbausteins in OB1

Die typische Steuerung des **LTMFAST_dosing_application**-Bausteins erfolgt durch das Eintragen azyklischer Daten in den zugehörigen Instanzdatenbaustein und das gleichzeitige Setzen der zyklischen Steuerungen für die gewünschten Kanäle. Ein Anwendungsbeispiel ist in Listing 4.8 dargestellt.

```

1 IF #LoadFlag = 1 THEN
2     #Channels [1].Load := 0;
3 END_IF;
4
5 IF #StartDQFlag = 1 THEN
6     #Channels [1].StartDQ := 0;
7 END_IF;
8
9 IF #Init = 0 THEN
10    #Init := 1;
11    #Channels [1].EnableDQ := 1;
12    "LTMFAST_dosing_application_DB".userDataContent [1] :=
13    16#3E8;
14    "LTMFAST_dosing_application_DB".userDataContent [15] :=
15    16#1;
16    #Channels [1].Load := 1;
17    #LoadFlag := 1;

```

```

18 END_IF;
19
20 IF ("FAST_FB".FB_IF_Channel[1] AND 16#8000) <> 0 THEN
21     #Channels[1].StartDQ := 1;
22     #StartDQFlag := 1;
23 END_IF;

```

Listing 4.8: Beispiel zur Steuerung der zyklischen und azyklischen Kommunikation

N2 – Funktionsumfang im Vergleich zur FM350-2

Alle für das Applikationsbeispiel *Dosieranwendung* relevanten Funktionen wurden erfolgreich umgesetzt. Einschränkungen bestehen lediglich bei einzelnen Funktionen wie der Drehzahlmessung, die zusätzliche Eingänge am Inkrementalgeber erfordert und daher in dieser Architektur nicht vorgesehen ist.

Tabelle 4.1 gibt eine Übersicht der implementierten und nicht implementierten Funktionen im Vergleich zur FM350-2. Dabei agieren **EnableDQ**, **Hold** und **StartDQ** als Software-Tore und stellen sicher, dass Zählimpulse zum Zähler gelangen oder gestoppt werden. Die Umsetzung eines Hardware-Tores war aufgrund eines zusätzlich benötigten Eingangs nicht möglich.

Funktion	Umsetzungsstatus
Torfunktionen der FM350-2	teilweise umgesetzt
Frequenzmessung	umgesetzt
Periodisches Zählen	umgesetzt
Einmaliges Zählen	nicht umgesetzt
Endloses Zählen	nicht umgesetzt
Prozessalarme	teilweise umgesetzt
Drehzahlmessung	nicht umgesetzt

Tabelle 4.1: Vergleich des Funktionsumfangs mit der FM350-2

Damit wird deutlich, dass alle für die Dosieranwendung notwendigen Kernfunktionen abgedeckt sind. Die teilweise umgesetzten Prozessalarme sind für die Anwendung relevant, werden jedoch in einem reduzierten Umfang realisiert. Nicht

umgesetzte Funktionen wie das einmalige Zählen, endlose Zählen oder die Drehzahlmessung betreffen dagegen Bereiche, die für die Dosierung keine wesentliche Rolle spielen.

N3 – Ressourcenschonung

Die FPGA-Auslastung wird durch Optimierungen reduziert. Abbildung 4.2 zeigt den Trend des Ressourcenverbrauchs.

Das VHDL-Design nutzt gezielt Arrays und Bitfelder, um den Ressourcenverbrauch im FPGA zu minimieren. Die Anzahl der tatsächlich verwendeten Kanäle wird zentral über das Generic ‘CHANNEL_COUNT‘ festgelegt (Lst. 4.1). Alle kanalbezogenen Signale (wie Zählerstände, Statusflags, Steuerbits) werden als Arrays der Länge ‘CHANNEL_COUNT‘ deklariert. Dadurch werden nur so viele Flipflops und Logikelemente instanziiert, wie für die aktivierten Kanäle tatsächlich benötigt werden.

Nicht verwendete Kanäle belegen somit keine unnötigen Ressourcen, da die Generate-Blöcke und Prozesse ausschließlich für die Indizes ‘0‘ bis ‘CHANNEL_COUNT-1‘ erzeugt werden. Auch die Kanaluordnung erfolgt über Mapping-Arrays, sodass keine redundante Logik für ungenutzte Ein-/Ausgänge entsteht.

Auch die Steuer- und Feedbackschnittstellen werden als Bitfelder (‘std_logic_vector‘) organisiert, sodass mehrere Steuer- und Statusinformationen platzsparend in einem einzigen Datenwort übertragen werden können. Dies reduziert den Bedarf an separaten Signalen und vereinfacht die Schnittstellenanbindung.

Flow Summary	
<<Filter>>	
Flow Status	Analyzed - Thu Sep 11 20:22:51 2025
Quartus Prime Version	23.1std.1 Build 993 05/14/2024 SC Standard Edition
Revision Name	TFL_MP_FAST_1
Top-level Entity Name	TFL_MP_FAST_1_TOP
Family	Cyclone 10 LP
Device	10CL025YU256A7G
Timing Models	Final
Total logic elements	12,928 / 24,624 (53 %)
Total registers	8186
Total pins	93 / 151 (62 %)
Total virtual pins	0
Total memory bits	Total virtual pins 08,256 (0 %)
Embedded Multiplier 9-bit elements	0 / 132 (0 %)
Total PLLs	1 / 4 (25 %)

Abbildung 4.2: Anzahl der Logikelemente bei einem Kanal

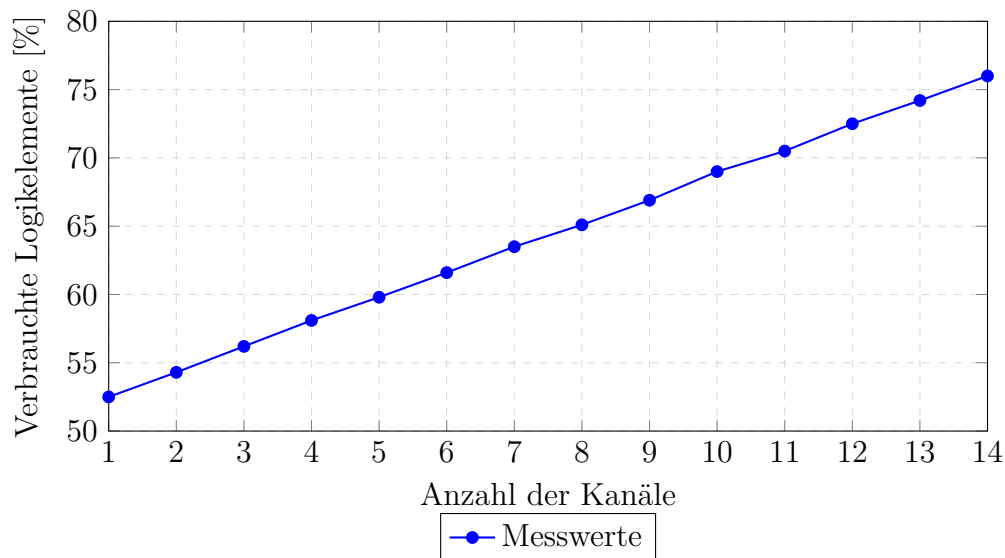


Abbildung 4.3: Prozentualer Verbrauch der Logikelemente in Abhängigkeit von der Kanalanzahl (Maximal 24 624 LEs)

Durch die konsequente Nutzung von Arrays, Bitfeldern und Generate-Strukturen wird der FPGA optimal ausgelastet und der Ressourcenverbrauch auf das tatsächlich benötigte Minimum reduziert. Für jeden nicht verwendeten Kanal werden im Mittel etwa 450 Logikelemente eingespart, was rund 1,8 % der Gesamtkapazität entspricht.

In Abbildung 4.3 ist erkennbar, dass bereits bei nur einem Kanal etwa 12 928 Logikelemente belegt sind, was rund 53 % der verfügbaren Ressourcen entspricht. Mit jedem zusätzlichen Kanal steigt die Auslastung linear um ungefähr 1,8 %. Dieser lineare Trend verdeutlicht die gute Skalierbarkeit der Architektur: Der Ressourcenverbrauch pro Kanal bleibt vergleichsweise gering, sodass auch Konfigurationen mit einer hohen Kanalanzahl effizient umgesetzt werden können, ohne die FPGA-Grenzen frühzeitig zu erreichen.

N4 – Verbesserte Reaktionszeit

Aufgrund der parallelen Verarbeitung in einem FPGA ist es möglich mehrere Funktionen gleichzeitig abzuarbeiten. Einziger limitierender Faktor ist die maximale I/O-Belegung der physischen Schnittstelle und die Anzahl der verfügbaren Logikelemente. Dadurch ist die Verarbeitungs- und Reaktionszeit der TM Fast mit 20ns deutlich schneller als die der FM 350-2 mit 50 μ s.

N5 – Erweiterbarkeit

Die modulare Architektur des Designs erlaubt eine einfache Erweiterung über die aktuell umgesetzten 14 Kanäle hinaus. Durch die Erhöhung der verfügbaren I/O-Schnittstellen kann die Kanalanzahl ohne Anpassung der Grundstruktur auf bis zu 25 Kanäle skaliert werden. Dabei bleibt der Ressourcenverbrauch dank des konstanten Aufwands von etwa 450 Logikelementen pro Kanal gut kalkulierbar.

Das vorgestellte VHDL-Design wurde gezielt mit Blick auf Erweiterbarkeit entwickelt. Zentrale Konzepte wie Generics, Array-Typen und Generate-Blöcke ermöglichen es, zusätzliche Kanäle oder neue Funktionalitäten einzubinden, ohne die bestehende Logik verändern zu müssen. Die Anzahl der Kanäle wird ausschließlich über die Konstante `CHANNEL_COUNT` festgelegt. Ein einfaches Anpassen dieses Werts sowie der Mapping-Arrays (`CHANNEL_MAP_IN_CH`, `CHANNEL_MAP_OUT_CH`) reicht aus, um das System auf eine größere Kanalanzahl zu erweitern.

Darüber hinaus ist die Architektur auch für funktionale Erweiterungen vorbereitet. Neue Messverfahren wie Frequenz- oder Impulsbreitenmessung können modular als zusätzliche Komponenten (z. B. `FB82_Freq32_e`) pro Kanal eingebunden werden. Die Steuer- und Feedbackschnittstellen sind als Bitfelder und Arrays ausgelegt, sodass weitere Steuerbits oder Statusinformationen ohne tiefgreifende Änderungen ergänzt werden können. Auch die Fehler- und Diagnoselogik ist pro Kanal gekapselt und lässt sich flexibel um zusätzliche Überwachungsmechanismen erweitern.

Durch die klare Trennung von Steuer-, Daten- und Diagnosepfaden sowie die konsequente Nutzung von Generate-Strukturen bleibt das Design flexibel, wartbar und zukunftssicher. Dies ermöglicht eine nachhaltige Weiterentwicklung der Applikation, ohne dass grundlegende Architekturänderungen erforderlich sind.

4.4 Validierung und Tests

Die Funktionalität des entwickelten VHDL-Designs wurde durch Simulationen und Tests umfassend nachgewiesen. Wellenformanalysen bestätigen die deterministische und zeitgenaue Funktionsweise des Systems.

Die Architektur ist gezielt auf eine hohe Testbarkeit ausgelegt. Durch die konsequente Trennung von Steuer- und Datenpfaden sowie die Nutzung von Arrays

und Generate-Blöcken kann jeder Kanal unabhängig simuliert und überprüft werden. Alle kanalbezogenen Signale sind als Arrays deklariert, sodass im Test gezielt einzelne Kanäle adressiert werden können, ohne unbeabsichtigte Seiteneffekte auf andere Kanäle auszulösen.

Die Steuer- und Feedbackschnittstellen sind klar voneinander getrennt und als Bitfelder organisiert. Dadurch lassen sich in der Simulation gezielt Steuerbits (z. B. `Load`, `StartDQ`, `CounterReset`) für einzelne Kanäle setzen und deren Reaktion im Design beobachten. Auch die Fehler- und Diagnoselogik ist pro Kanal gekapselt, sodass Fehlerbedingungen (z. B. `DI_QI_BAD`, `DQ_QI_BAD`) gezielt getestet werden können.

Die resultierenden Signalverläufe wurden in *Questa* aufgezeichnet. Abbildung 4.4 zeigt einen Ausschnitt dieser Wellenanalyse, in der sowohl die zyklischen Steuerbits als auch die azyklischen Schnittstellen `WR_REC` und `RD_REC` sichtbar sind.

Gut erkennbar ist, dass die zyklischen Steuerbits (z. B. `CTRL_IF0`, `CTRL_IF1`) in jedem SPS-Zyklus gesetzt und über das Prozessabbild verarbeitet werden. Parallel dazu laufen azyklische Operationen über die `WR_REC`- und `RD_REC`-Schnittstellen ab, die nur bei Bedarf aktiviert werden.

Im dargestellten Beispiel wird ein `LoadRequest` über die `WR_REC`-Schnittstelle angestoßen. Dieser führt dazu, dass die entsprechenden Steuerbits aktiv werden und anschließend im Zähler `j_enc_count` Änderungen auftreten. Auf diese Weise lässt sich in der Simulation nachvollziehen, wie deterministisch die zyklischen Steuerbefehle verarbeitet und die azyklische Kommandos synchron eingesteuert werden.

5 Fazit und Ausblick

Im Rahmen dieser Arbeit konnte gezeigt werden, dass das FM350-2 Modul der SIMATIC S7-300 erfolgreich durch ein VHDL-basiertes Applikationsbeispiel auf dem TM FAST Modul der SIMATIC S7-1500 ersetzt werden kann. Alle für Dosieranlagen relevanten Funktionen wurden umgesetzt und in das neue Design integriert.

Ein zentrales Ergebnis ist die signifikante Verkürzung der Reaktionszeit: Während die FM350-2 Baugruppe eine Verzögerung von rund 50 μ s aufwies, erreicht die neue Implementierung eine Zykluszeit von 20 ns. Dadurch wird eine sofortige und deterministische Signalverarbeitung ermöglicht, was die Eignung für zeitkritische Dosierprozesse deutlich erhöht.

Darüber hinaus konnte die Kanalanzahl von ursprünglich 8 auf 14 im TM FAST Modul erweitert werden. Dies eröffnet neue Möglichkeiten für den parallelen Betrieb mehrerer Dosierkanäle und steigert sowohl die Flexibilität als auch die Effizienz der Anwendung.

Auch die Ressourcennutzung wurde optimiert: Durch gezielte Anpassungen im VHDL-Design konnte die Auslastung der FPGA-Logikelemente auf maximal 80 % begrenzt werden, sodass Reserven für zukünftige Erweiterungen bestehen.

Schließlich wurde das Applikationsbeispiel vollständig in das TIA Portal integriert. Damit steht eine nahtlose Einbindung in bestehende Engineering-Umgebungen zur Verfügung, was die Inbetriebnahme und Diagnose im industriellen Einsatz erheblich erleichtert.

Insgesamt belegt die Arbeit, dass das TM FAST Modul eine leistungsfähige und zukunftssichere Alternative zum FM350-2 darstellt. Die erfolgreiche Umsetzung der funktionalen und nicht-funktionalen Anforderungen schafft die Grundlage für den Einsatz moderner FPGA-Technologie in hochkanaligen Dosieranwendungen und verdeutlicht deren Potenzial für die industrielle Automatisierung.

5.1 Kritische Reflexion und Herausforderungen

Ein zentrales Problem bestand darin, dass kein realer Aufbau der alten FM 350-2-Baugruppe verfügbar war. Vergleichsdaten über die Zykluszeit mussten daher ausschließlich aus der Dokumentation und dem Handbuch entnommen werden. Dadurch konnten bestimmte Messungen oder Tests zum Vergleich der Reaktionszeiten und Fehlerdiagnosen nicht durchgeführt werden. Diese Einschränkung führte dazu, dass die Analyse an diesen Teilen theoretisch blieb und auf Annahmen basierte, die nicht vollständig verifiziert werden konnten.

5.2 Potenzielle Weiterentwicklungen und zukünftige Anwendungen

Ein wichtiger Ansatzpunkt für zukünftige Arbeiten ist die Integration von Prozessalarmen, wie sie in der FM 350-2 vorgesehen waren. Diese könnten über eine geeignete Erweiterung des azyklischen Datensatzes `RD_REC` realisiert werden, um sowohl detaillierte Fehlerinformationen als auch Diagnosehinweise sicher zu übertragen. Ziel sollte es sein, die Vorteile der neuen Architektur – insbesondere die extrem kurze Reaktionszeit und die sichere Fehlerbehandlung – mit der umfangreichen Diagnosetiefe der alten Baugruppe zu kombinieren.

Darüber hinaus bietet das flexible VHDL-Design eine solide Grundlage für weitere Applikationen. Durch die Anpassung der Kanalzuordnung und die Erweiterung der Kommunikationsschnittstellen können neue Einsatzfelder erschlossen werden, beispielsweise für hochpräzise Dosier- oder Zählsysteme in unterschiedlichen industriellen Bereichen. Auch die Anbindung an moderne Steuerungssysteme über standardisierte Protokolle stellt ein mögliches Entwicklungsfeld dar, um die Integration in komplexe Automatisierungsumgebungen weiter zu vereinfachen.

Abbildungsverzeichnis

2.1	Aufbau eines SIMATIC S7-Systems [9]	4
2.2	Zentraler und dezentraler Aufbau der SIMATIC S7-1500 Steuerung mit ET 200MP Peripheriesystem [9]	6
2.3	Hardwarekonfiguration im TIA Portal mit TM FAST	8
2.4	Zeitdiagramm zur Simulation einer Signum-Funktion in der Questa- Intel® FPGA Starter Edition	10
2.5	Zwei Rechteckwellen in Quadratur [12]	11
2.6	Funktionsweise eines Inkrementalgebers [12]	11
3.1	Hardware der FM-350-2 [2]	14
3.2	Endloses Zählen in Hauptzählrichtung vorwärts [2]	15
3.3	Einmaliges Zählen in Hauptzählrichtung vorwärts [2]	16
3.4	Periodisches Zählen in Hauptzählrichtung vorwärts [2]	17
3.5	Prinzip der Frequenzmessung mit Zeitfenster bei der FM 350-2 [2] .	18
3.6	Öffnen und Schließen eines Tores und das Zählen der Impulse [2] .	19
3.7	Blockdiagramm des TM FAST [11]	22
3.8	Aufbau von Frontend- und Backend-Gerät und Einbindung in das System [11]	23
3.9	Technologiemodul TM FAST [3]	24
4.1	Integration des Applikationsbausteins in OB1	39
4.2	Anzahl der Logikelemente bei einem Kanal	41
4.3	Prozentualer Verbrauch der Logikelemente in Abhängigkeit von der Kanalanzahl (Maximal 24 624 LEs)	42
4.4	Wellenanalyse der CTRL_IF-, WR_REC- und RD_REC-Schnittstellen in Questa	45

Tabellenverzeichnis

3.1	Kennzeichnungen und Komponenten der FM 350-2 [2]	15
3.2	Zusammenhang der Torfunktionen und des Zählvorgangs [2]	19
3.3	Ressourcen des Intel® Cyclone 10 LP FPGA [10]	23
3.4	Kompakter Vergleich der Eigenschaften von TM FAST und FM 350-2 (Daten gemäß [3, 9, 2]).	25
4.1	Vergleich des Funktionsumfangs mit der FM350-2	40

Literaturverzeichnis

- [1] Liam Bee (2022): *PLC and HMI development with Siemens TIA Portal: develop PLC and HMI programs using standard methods and structured approaches with TIA Portal V17*. Packt Publishing, Limited.
- [2] Siemens AG (2011): *Zählerbaugruppe FM 350-2 Gerätehandbuch*. Verfügbar unter: https://cache.industry.siemens.com/dl/files/178/1105178/att_20226/v1/s7300_fm350_2_operating_instructions_de_de-DE.pdf. Zugriff: 11. März 2025.
- [3] Siemens AG (2023): *TMFast Programmeier Handbuch*. Verfügbar unter: https://cache.industry.siemens.com/dl/files/088/109816088/att_1144842/v2/s71500_tm_fast_programming_manual_de-DE_de-DE.pdf. Zugriff: 12. März 2025.
- [4] Wolfgang Bable: *Die Automatisierungspyramide von 1985 – Grundlegende Struktur in IoT und Industrie 4.0*
- [5] H. Berger: *Automatisieren mit SIMATIC S7-1500: Projektieren, Programmieren und Testen mit STEP 7 Professional*. 2. Aufl. Erlangen: Publics Publishing, 2017.
- [6] W. Babel: *Systemintegration in Industrie 4.0 und IoT: Vom Ethernet bis hin zum Internet und OPC UA*. Wiesbaden: Springer Fachmedien, 2024. DOI: <https://doi.org/10.1007/978-3-658-42987-4>.
- [7] Siemens AG: *Totally Integrated Automation Portal*. siemens.com. Verfügbar unter: <https://new.siemens.com/de/de/produkte/automatisierung/industrie-software/automatisierungs-software/tia-portal.html> (abgerufen am 26.07.2025).
- [8] Siemens AG: *SIMATIC Energy Suite*. siemens.com. Verfügbar unter: <https://www.siemens.com/de/de/produkte/automatisierung/>

- [industrie-software/automatisierungs-software/energiemanagement/simatic-energy-suite.html](#) (abgerufen am 26.07.2025).
- [9] Siemens AG: *SIMATIC ET 200MP*. siemens.com. Verfügbar unter: <https://new.siemens.com/de/de/produkte/automatisierung/systeme/industrie/io-systeme/simatic-et-200mp.html> (abgerufen am 26.07.2025).
- [10] Intel® Corp.: *Intel Cyclone 10 LP Device Overview*, 2022. [Online] Verfügbar unter: <https://www.intel.com/content/www/us/en/docs/programmable/683879/current/device-overview.html> (abgerufen am 27.07.2025).
- [11] G. Wild *et al.*: *Feature Specification for TM FAST*, Version 2.0, 2025.
- [12] Encoder Products Company: *The Basics of How an Encoder Works*, White Paper (WP-2011), rev. 15/08/25, 2025. [Online] Verfügbar unter: https://www.encoder.com/hubfs/white-papers/WP-2011_Basics/wp2011-basics-how-an-encoder-works.pdf (abgerufen am 15. August 2025).
- [13] Sensoray, Inc.: *Introduction to Incremental Encoders*, Application Note. [Online] Verfügbar unter: <https://www.sensoray.com/support/appnotes/encoders.htm> (abgerufen am 15. August 2025).
- [14] JUMO GmbH & Co. KG: *Regelungstechnik – Grundlagen und Anwendungen*. Verfügbar unter: https://www.spsHaus.ch/files/inc/Downloads/Lernumgebung/Downloads/Allgemein/Regelungstechnik_Jumo.pdf (abgerufen am 03. September 2025).
- [15] Prof. Dr.-Ing. K. Stephan: *Automatisierung in der Prozesstechnik – Lehrstoffsammlung, Kapitel I*. Verfügbar unter: <https://automatisierung-stephan.de/wp-content/uploads/2024/07/Automatisierung-in-der-Prozesstechnik-I.pdf> (abgerufen am 05. September 2025).
- [16] Hering, E.; Endres, J.; Gutekunst, J. (Hrsg.): *Elektronik für Ingenieure und Naturwissenschaftler*. Springer Vieweg, Wiesbaden.
- [17] Siemens AG: *Function Blocks, OBs, FCs, and DBs in SIMATIC S7-1200 — Programming Concepts*. Online-Dokumentation. Verfügbar unter: https://docs.tia.siemens.cloud/r/simatic_s7_1200_manual_collection_enus_

- [20/programming-concepts/using-blocks-to-structure-your-program/function-block-fb](#) (abgerufen am 05. September 2025).
- [18] Heinrich, B.; Linke, P.; Glöckler, M.: *Grundlagen Automatisierung*. Hanser Verlag, München, 2018.
- [19] Siemens AG: *High Speed Boolean Processor FM 352-5*. Verfügbar unter: <https://mall.industry.siemens.com/mall/de/b1/Catalog/Products/10000781?activeTab=product> (abgerufen am 06. September 2025).
- [20] Intel® Corporation: *Intel Quartus Prime Standard Edition Handbook Volume 1: Design and Compilation*, 2023. [Online] Verfügbar unter: <https://www.intel.com/content/www/us/en/docs/programmable/683082/current/overview.html> (abgerufen am 27.07.2025).
- [21] Intel® Corporation: *Intel Quartus Prime Software*. [Online] Verfügbar unter: <https://www.intel.de/content/www/de/de/products/details/fpga/development-tools/quartus-prime.html> (abgerufen am 27.07.2025).
- [22] Intel® Corporation: *Questa*-Intel® FPGA Edition Software*. [Online] Verfügbar unter: <https://www.intel.de/content/www/de/de/software/programmable/quartus-prime/questa-edition.html> (abgerufen am 27.08.2025).
- [23] Siemens AG: *SIMATIC S7-1500 Structure and Use of the CPU Memory Function Manual*. Version 01/2013, Dokument-Nr. A5E03461664-01. [PDF] Verfügbar unter: https://cache.industry.siemens.com/dl/files/101/59193101/att_80162/v1/s71500_structure_and_use_of_the_PLC_memory_function_manual_en-US_en-US.pdf (abgerufen am 9.09.2025).
- [24] Siemens AG: *SIMATIC Safety Integrated – machine safety seamlessly integrated*. Webseite. Verfügbar unter: <https://www.siemens.com/global/en/products/automation/topic-areas/safety-integrated/factory-automation/offering/simatic-safety.html> (abgerufen am 06. September 2025).
- [25] Siemens AG: *SIMATIC S7-300 – Bewährt und verfügbar bis 2033*. Webseite. Verfügbar unter: <https://www.siemens.com/de/de/produkte/automatisierung/systeme/industrie/sps/simatic-s7-300.html> (abgerufen am 11. September 2025).

- [26] An automated and parallelised DIY-dosing unit for individual and complex feeding profiles: Construction, validation and applications. Webseite. Verfügbar unter: <https://pmc.ncbi.nlm.nih.gov/articles/PMC6583958> (abgerufen am 11. September 2025).